

```
// ===== 前 30 页连续源码 =====
// ===== 文件: lib/main.dart =====
0001 import 'dart:ui';
0002 import 'package:flutter/material.dart';
0003 import 'package:flutter/services.dart';
0004 import 'core/database/storage_service.dart';
0005 import 'core/theme/cyber_colors.dart';
0006 import 'core/utils/cyber_logger.dart';
0007 import 'features/settings/presentation/widgets/privacy_agreement_modal.dart';
0008 import 'features/audio/services/ambient_service.dart';
0009 import 'features/audio/services/sfx_service.dart';
0010 import 'features/dashboard/presentation/dashboard_screen.dart';
0011 import 'features/library/domain/book_model.dart';
0012 import 'features/reader/providers/reader_provider.dart';
0013 import 'features/settings/providers/settings_provider.dart';
0014 import 'package:flutter_riverpod/flutter_riverpod.dart' as riverpod;
0015 import 'shared/widgets/tts_error_listener.dart';
0016 import 'features/audio/services/tts_engine_service.dart';
0018 final GlobalKey<NavigatorState> globalNavigatorKey =
0019     GlobalKey<NavigatorState>();
0021 Future<void> main() async {
0022     WidgetsFlutterBinding.ensureInitialized();
0024     // 全局错误捕获锚点（先注册，确保 Sentry 初始化前的错误也能记录）
0025     FlutterError.onError = CyberLogger.recordFlutterError;
0026     PlatformDispatcher.instance.onError = CyberLogger.recordPlatformError;
0028     // 启动前初始化持久化层与音效引擎
0029     await StorageService.init();
0030     try {
0031         await SfxService.init();
0032     } catch (e, st) {
0033         CyberLogger.recordFlutterError(
0034             FlutterErrorDetails(exception: e, stack: st, library: 'SfxService'),
0035         );
0036     }
0038     // 初始化环境背景音服务
0039     try {
0040         await AmbientService.init();
0041     } catch (e, st) {
0042         CyberLogger.recordFlutterError(
0043             FlutterErrorDetails(exception: e, stack: st, library: 'AmbientService'),
0044         );
0045     }
0047     // 通过 CyberLogger 初始化 Sentry 并启动 App
0048     // DSN 通过 --dart-define=SENTRY_DSN=https://... 注入，空时静默跳过
0049     await CyberLogger.initSentry(
0050         () async => runApp(
0051             const riverpod.ProviderScope(
0052                 child: YueYouApp(),
0053             ),
0054         ),
```

```
0055 );
0056 }
0057 class YueYouApp extends riverpod.ConsumerWidget {
0058   const YueYouApp({super.key});
0059   @override
0060   Widget build(BuildContext context, riverpod.WidgetRef ref) {
0061     return const _Bootstrapper();
0062   }
0063 }
0064 class _Bootstrapper extends riverpod.ConsumerStatefulWidget {
0065   const _Bootstrapper();
0066   @override
0067   riverpod.ConsumerState<_Bootstrapper> createState() => _BootstrapperState();
0068 }
0069 class _BootstrapperState extends riverpod.ConsumerState<_Bootstrapper>
0070   with WidgetsBindingObserver {
0071   bool _booted = false;
0072   /// 记录上一次 SettingsProvider 的 ambient 状态，用于增量对比
0073   bool? _lastAmbientEnabled;
0074   double? _lastAmbientVol;
0075   String? _lastAmbientStyle;
0076   @override
0077   void initState() {
0078     super.initState();
0079     WidgetsBinding.instance
0080       ..addObserver(this)
0081       ..addPostFrameCallback((_) => _checkPrivacyAndBootstrap());
0082   }
0083   @override
0084   void didChangeDependencies() {
0085     super.didChangeDependencies();
0086     // 每当 SettingsProvider 变化时同步 AmbientService
0087     final settings = ref.watch(settingsProvider);
0088     _syncAmbient(settings);
0089   }
0090   /// 同步 SettingsProvider 的 ambient 设置到 AmbientService
0091   void _syncAmbient(SettingsProvider settings) {
0092     if (settings.ambientEnabled != _lastAmbientEnabled) {
0093       _lastAmbientEnabled = settings.ambientEnabled;
0094       AmbientService.setEnabled(settings.ambientEnabled);
0095     }
0096     if (settings.ambientVol != _lastAmbientVol) {
0097       _lastAmbientVol = settings.ambientVol;
0098       AmbientService.setVolume(settings.ambientVol);
0099     }
0100     if (settings.ambientStyle != _lastAmbientStyle) {
0101       _lastAmbientStyle = settings.ambientStyle;
0102       AmbientService.setStyle(settings.ambientStyle);
0103     }
0104   }
0105 }
```

```
0115  /// App 生命周期: 切后台暂停, 切前台恢复
0116  @override
0117  void didChangeAppLifecycleState(AppLifecycleState state) {
0118      switch (state) {
0119          case AppLifecycleState.paused:
0120          case AppLifecycleState.inactive:
0121              AmbientService.pause();
0122          case AppLifecycleState.resumed:
0123              AmbientService.resume();
0124          case AppLifecycleState.detached:
0125          case AppLifecycleState.hidden:
0126              AmbientService.dispose();
0127      }
0128  }
0129
0130  Future<void> _checkPrivacyAndBootstrap() async {
0131      if (!mounted) return;
0132      if (!StorageService.hasAgreedPrivacy()) {
0133          // 等待 MaterialApp 内部 Navigator 挂载完成 (最多 5 次, 间隔 50ms)
0134          // 避免首帧时 globalNavigatorKey 尚未绑定导致弹窗被静默跳过
0135          BuildContext? navContext;
0136          for (int i = 0; i < 5; i++) {
0137              navContext = globalNavigatorKey.currentContext;
0138              if (navContext != null) break;
0139              await Future<void>.delayed(const Duration(milliseconds: 50));
0140              if (!mounted) return;
0141          }
0142          if (navContext == null) {
0143              // 极端情况: Navigator 始终未挂载, 记录日志后退出, 绝不进入未授权状态
0144              CyberLogger.captureWarning(
0145                  Exception('隐私弹窗 navContext 重试 5 次仍为 null'),
0146                  tag: 'privacy',
0147                  extra: {'context': '_checkPrivacyAndBootstrap'},
0148              );
0149              SystemNavigator.pop();
0150              return;
0151          }
0152          // navContext 来自 globalNavigatorKey, 由 Navigator 自身管理生命周期,
0153          // 不依赖 _BootstrapperState.mounted; 上方循环已做 mounted 守卫。
0154          // ignore: use_build_context_synchronously
0155          final agreed = await showPrivacyAgreementModal(navContext);
0156          if (!mounted) return;
0157          if (agreed) {
0158              await StorageService.setHasAgreedPrivacy(true);
0159          } else {
0160              return;
0161          }
0162      }
0163      await _bootstrap();
0164  }
0165
0166  Future<void> _bootstrap() async {
```

```

0167     if (_booted) return;
0168     _booted = true;
0170     final reader = ref.read(readerProvider);
0171     final currentNovelId = StorageService.getCurrentNovelId();
0172     if (currentNovelId == null) return;
0174     final content = await StorageService.loadBookContent(currentNovelId);
0175     if (!mounted || content == null) return;
0177     final List<dynamic> rawLines = content['lines'] as List<dynamic>? ?? [];
0178     final List<dynamic> rawChapters =
0179         content['chapters'] as List<dynamic>? ?? [];
0180     final chapters = rawChapters
0181         .map((e) => ChapterModel.fromJson(e as Map<String, dynamic>))
0182         .toList();
0183     final lines = rawLines.map((e) => e.toString()).toList();
0184     final int initialIndex = StorageService.getCurrentNovelIndex();
0186     await reader.loadPreparedBook(
0187         lines,
0188         bookId: currentNovelId,
0189         chapters: chapters,
0190         initialIndex: initialIndex,
0191     );
0192 }
0194 @override
0195 void dispose() {
0196     WidgetsBinding.instance.removeObserver(this);
0197     super.dispose();
0198 }
0200 @override
0201 Widget build(BuildContext context) {
0202     return MaterialApp(
0203         navigatorKey: globalNavigatorKey,
0204         title: '阅游 YueYou',
0205         debugShowCheckedModeBanner: false,
0206         theme: ThemeData(
0207             scaffoldBackgroundColor: CyberColors.background,
0208             colorScheme: const ColorScheme.dark(
0209                 primary: CyberColors.neonGreen,
0210                 secondary: CyberColors.neonPink,
0211             ),
0212             useMaterial3: true,
0213         ),
0214         builder: (context, child) => Listener(
0215             onPointerDown: (_) => ref.read(ttsEngineProvider).notifyUserActivity(),
0216             behavior: HitTestBehavior.translucent,
0217             child: TtsErrorListener(child: child!),
0218         ),
0219         home: const DashboardScreen(),
0220     );
0221 }
0222 }

```

```
// ===== 文件结束: lib/main.dart =====

// ===== 文件: lib/core/config/app_info_config.dart =====
0001 /// 应用主体信息配置。
0002 ///
0003 /// 开发者/运营主体和联系方式支持通过 `--dart-define` 覆盖。
0004 /// 默认值为当前个人开发者公开信息。
0005 class AppInfoConfig {
0006   const AppInfoConfig._();
0008   /// 运营主体名称。
0009   static const String developerName = String.fromEnvironment(
0010     'APP_DEVELOPER_NAME',
0011     defaultValue: '胡传龙',
0012   );
0014   /// 隐私与用户反馈联系邮箱。
0015   static const String contactEmail = String.fromEnvironment(
0016     'APP_CONTACT_EMAIL',
0017     defaultValue: 'hucloong@163.com',
0018   );
0020   /// 正式隐私政策页面。
0021   static const String privacyPolicyUrl = String.fromEnvironment(
0022     'PRIVACY_POLICY_URL',
0023     defaultValue: 'https://hclstudio.cn/privacy',
0024   );
0025 }

// ===== 文件结束: lib/core/config/app_info_config.dart =====

// ===== 文件: lib/core/config/tts_config.dart =====
0001 /// TTS 配置类
0002 ///
0003 /// 所有服务器地址通过 `--dart-define` 编译时注入，严禁硬编码 IP 或域名。
0004 class TtsConfig {
0005   final String serverUrl;
0006   final Duration requestTimeout;
0007   final int maxRetries;
0008   final Duration baseRetryDelay;
0009   final int maxPrefetchQueue;
0011   const TtsConfig({
0012     required this.serverUrl,
0013     this.requestTimeout = const Duration(seconds: 8),
0014     this.maxRetries = 2,
0015     this.baseRetryDelay = const Duration(milliseconds: 800),
0016     this.maxPrefetchQueue = 6,
0017   });
0019   /// 编译时注入的 TTS 服务器地址 (--dart-define=TTS_SERVER_URL=https://...)
0020   static const String _ttsServerUrl = String.fromEnvironment(
0021     'TTS_SERVER_URL',
0022     defaultValue: 'https://hclstudio.cn/api/v1/tts',
0023   );
0025   /// 编译时注入的书籍 API 基础地址 (--dart-define=BOOK_API_BASE=https://...)
```

```
0026 static const String bookApiBase = String.fromEnvironment(
0027     'BOOK_API_BASE',
0028     defaultValue: 'https://hclstudio.cn/api/v1',
0029 );
0031 // —— 网络层超时常量 ——
0033 /// TTS 本地引擎朗读最长等待时间（防止 FlutterTts 卡死）
0034 static const Duration ttsLocalSpeakTimeout = Duration(seconds: 60);
0036 /// TTS HTTP 下载连接与传输超时
0037 static const Duration ttsDownloadTimeout = Duration(seconds: 15);
0039 /// TTS POST 请求连接超时
0040 static const Duration ttsPostConnectionTimeout = Duration(seconds: 10);
0042 /// TTS POST 响应超时
0043 static const Duration ttsPostResponseTimeout = Duration(seconds: 15);
0045 /// 书籍目录 / 章节 API 请求超时
0046 static const Duration bookApiTimeout = Duration(seconds: 4);
0048 /// 书籍章节 CDN 下载超时
0049 static const Duration bookCdnDownloadTimeout = Duration(seconds: 15);
0051 /// 当前环境配置（编译时确定，零运行时开销）
0052 static const TtsConfig current = TtsConfig(
0053     serverUrl: _ttsServerUrl,
0054 );
0055 }

// ===== 文件结束: lib/core/config/tts_config.dart =====

// ===== 文件: lib/core/constants/book_constants.dart =====
0001 /// 书籍相关常量定义
0002 class BookConstants {
0003     /// 内置默认书籍 ID（西游记），int 类型对齐 BookModel.id
0004     static const int defaultBookId = 999;
0006     /// 内置默认书籍标识字符串（用于分章缓存目录命名）
0007     static const String defaultBookKey = 'xiyouji';
0009     /// 内置默认书籍标题
0010     static const String defaultBookTitle = '西游记';
0012     /// 内置默认书籍作者
0013     static const String defaultBookAuthor = '吴承恩';
0015     /// 内置默认书籍总章节数
0016     static const int defaultTotalChapters = 100;
0018     /// 《西游记》100 回章节标题（离线降级用，索引 0-based 对应第1回至第100回）
0019     static const List<String> xiyoujiChapterTitles = [
0020         '第一回 灵根育孕源流出 心性修持大道生',
0021         '第二回 悟彻菩提真妙理 断魔归本合元神',
0022         '第三回 四海千山皆拱伏 九幽十类尽除名',
0023         '第四回 官封弼马心何足 名注齐天意未宁',
0024         '第五回 乱蟠桃大圣偷丹 反天宫诸神捉怪',
0025         '第六回 观音赴会问原因 小圣施威降大圣',
0026         '第七回 八卦炉中逃大圣 五行山下定心猿',
0027         '第八回 我佛造经传极乐 观音奉旨上长安',
0028         '第九回 袁守诚妙算无私曲 老龙王拙计犯天条',
0029         '第十回 二将军宫门镇鬼 唐太宗地府还魂',
0030         '第十一回 还受生唐王遵善果 度孤魂萧瑀正空门',
```

0031 '第十二回 玄奘秉诚建大会 观音显象化金蝉',
 0032 '第十三回 陷虎穴金星解厄 双叉岭伯钦留僧',
 0033 '第十四回 心猿归正 六贼无踪',
 0034 '第十五回 蛇盘山诸神暗佑 鹰愁涧意马收缰',
 0035 '第十六回 观音院僧谋宝贝 黑风山怪窃袈裟',
 0036 '第十七回 孙行者大闹黑风山 观世音收伏熊罴怪',
 0037 '第十八回 观音院唐僧脱难 高老庄大圣除魔',
 0038 '第十九回 云栈洞悟空收八戒 浮屠山玄奘受心经',
 0039 '第二十回 黄风岭唐僧有难 半山中八戒争先',
 0040 '第二十一回 护法设庄留大圣 须弥灵吉定风魔',
 0041 '第二十二回 八戒大战流沙河 木叉奉法收悟净',
 0042 '第二十三回 三藏不忘本 四圣试禅心',
 0043 '第二十四回 万寿山大仙留故友 五庄观行者窃人参',
 0044 '第二十五回 镇元仙赶捉取经僧 孙行者大闹五庄观',
 0045 '第二十六回 孙悟空三岛求方 观世音甘泉活树',
 0046 '第二十七回 尸魔三戏唐三藏 圣僧恨逐美猴王',
 0047 '第二十八回 花果山群妖聚义 黑松林三藏逢魔',
 0048 '第二十九回 脱难江流来国土 承恩八戒转山林',
 0049 '第三十回 邪魔侵正法 意马忆心猿',
 0050 '第三十一回 猪八戒义激猴王 孙行者智降妖怪',
 0051 '第三十二回 平顶山功曹传信 莲花洞木母逢灾',
 0052 '第三十三回 外道迷真性 元神助本心',
 0053 '第三十四回 魔王巧算困心猿 大圣腾挪骗宝贝',
 0054 '第三十五回 外道施威欺正性 心猿获宝伏邪魔',
 0055 '第三十六回 心猿正处诸缘伏 劈破旁门见月明',
 0056 '第三十七回 鬼王夜谒唐三藏 悟空神化引婴儿',
 0057 '第三十八回 婴儿问母知邪正 金木参玄见假真',
 0058 '第三十九回 一粒金丹天上得 三年故主世间生',
 0059 '第四十回 婴儿戏化禅心乱 猿马刀圭木母空',
 0060 '第四十一回 心猿遭火败 木母被魔擒',
 0061 '第四十二回 大圣殷勤拜南海 观音慈善缚红孩',
 0062 '第四十三回 黑河妖孽擒僧去 西洋龙子捉鼉回',
 0063 '第四十四回 法身元运逢车力 心正妖邪度脊关',
 0064 '第四十五回 三清观大圣留名 车迟国猴王显法',
 0065 '第四十六回 外道弄强欺正法 心猿显圣灭诸邪',
 0066 '第四十七回 圣僧夜阻通天水 金木垂慈救小童',
 0067 '第四十八回 魔弄寒风飘大雪 僧思拜佛履层冰',
 0068 '第四十九回 三藏有灾沉水宅 观音救难现鱼篮',
 0069 '第五十回 情因旧恨生灾毒 心主遭魔幸破光',
 0070 '第五十一回 心猿空用千般计 水火无功难炼魔',
 0071 '第五十二回 悟空大闹金兜洞 如来暗示主人公',
 0072 '第五十三回 禅主吞餐怀鬼孕 黄婆运水解邪胎',
 0073 '第五十四回 法性西来逢女国 心猿定计脱烟花',
 0074 '第五十五回 色邪淫戏唐三藏 性正归真胜女色',
 0075 '第五十六回 神狂诛草寇 道昧放心猿',
 0076 '第五十七回 真行者落伽山诉苦 假猴王水帘洞誉文',
 0077 '第五十八回 二心搅乱大乾坤 一体难修真寂灭',
 0078 '第五十九回 唐三藏路阻火焰山 孙行者一调芭蕉扇',
 0079 '第六十回 牛魔王罢战赴华筵 孙行者二调芭蕉扇',
 0080 '第六十一回 猪八戒助力败魔王 孙行者三调芭蕉扇',

```

0081      '第六十二回 涤垢洗心惟扫塔 缚魔归正乃修身',
0082      '第六十三回 二僧荡涤波月洞 胜法降伏无底船',
0083      '第六十四回 荆棘岭悟能努力 木仙庵三藏谈诗',
0084      '第六十五回 妖邪假设小雷音 四众皆遭大厄难',
0085      '第六十六回 诸神遭毒手 弥勒缚妖魔',
0086      '第六十七回 拯救驼罗庄 消除疥癬病',
0087      '第六十八回 朱紫国唐僧论前世 孙行者施为三折肱',
0088      '第六十九回 心主夜间修药物 君王筵上论妖邪',
0089      '第七十回 妖魔宝放烟沙火 悟空计盗紫金铃',
0090      '第七十一回 行者假名降怪狻 观音现像伏金毛',
0091      '第七十二回 盘丝洞七情迷本 濯垢泉八戒忘形',
0092      '第七十三回 情因旧恨盘丝洞 壁脱香风蜈蚣精',
0093      '第七十四回 长庚传报魔头狠 行者施为变化能',
0094      '第七十五回 心猿钻入魔王腹 将法三番数字成',
0095      '第七十六回 心神居舍魔归性 木母同降怪体真',
0096      '第七十七回 群魔欺本性 一体拜真如',
0097      '第七十八回 比丘怜子遣阴神 金殿识魔谈道德',
0098      '第七十九回 寻洞擒妖逢老寿 当朝正主救婴儿',
0099      '第八十回 姹女育阳求配偶 心猿护主识妖邪',
0100      '第八十一回 镇海寺心猿知怪 黑松林三众寻师',
0101      '第八十二回 姹女求阳 元神护道',
0102      '第八十三回 心猿识得丹头 姹女还归本性',
0103      '第八十四回 难灭伽持圆大觉 法王成正体天然',
0104      '第八十五回 心猿妒木母 魔主计吞禅',
0105      '第八十六回 木母助威征怪物 金公施法灭妖邪',
0106      '第八十七回 凤仙郡冒天止雨 孙大圣劝善施霖',
0107      '第八十八回 禅到玉华施法会 心猿木母授门人',
0108      '第八十九回 黄狮精虎力大 白鹿精太乙真',
0109      '第九十回 师狮授受同归一 盗宝欺人反受殃',
0110      '第九十一回 金平府元夜观灯 玄英洞唐僧供状',
0111      '第九十二回 三僧大战青龙山 四星挟捉犀牛怪',
0112      '第九十三回 给孤园问古怀乡 天竺国朝王遇偶',
0113      '第九十四回 四僧宴乐御花园 一怪空怀情欲喜',
0114      '第九十五回 假合真形擒玉兔 真阴归正会灵元',
0115      '第九十六回 寇员外喜待高僧 唐长老不贪富贵',
0116      '第九十七回 金酬外护遭魔蛰 圣显幽魂救本原',
0117      '第九十八回 猿熟马驯方论正 功完行满见真如',
0118      '第九十九回 九九数完魔灭尽 三三行满道归根',
0119      '第一百回 径回东土 五圣成真',

```

```
0120 ];
```

```
0121 }
```

```
// ===== 文件结束: lib/core/constants/book_constants.dart =====
```

```
// ===== 文件: lib/core/constants/cyber_error_messages.dart =====
```

```

0001 class CyberErrorMessages {
0002     // =====
0003     // 网络与云端通讯错误（屏蔽底层实现细节）
0004     // =====
0005     static const String ttsNodeUnresponsive = '语音服务暂不可用，请检查网络后重试';
0006     static const String ttsConnectTimeout = '服务连接超时，请检查网络设置';

```



```
0007 static const String ttsRequestTimeout = '请求处理超时, 请稍后重试';
0008 static const String ttsAudioLoadTimeout = '语音加载缓慢, 为您切换至本地语音';
0010 static String ttsServerErrorCode(int code) => '服务器繁忙, 请稍后重试 (错误码: $code)';
0012 // =====
0013 // 降级与兜底提醒 (用户友好的状态变更)
0014 // =====
0015 static const String ttsFallbackDisconnected = '云端连接因网络波动断开, 已自动切换为本地语音';
0016 static const String ttsFallbackTimeout = '云端资源加载失败, 正在使用本地语音播放';
0018 // =====
0019 // 协议与解析异常 (隐藏服务架构与堆栈)
0020 // =====
0021 static const String ttsInvalidFormat = '语音数据异常, 请确保应用为最新版或稍后重试';
0022 static const String ttsNotJsonObject = '语音服务暂不支持该解析, 请稍后重试';
0024 static String ttsMissingUrl(String body) => '获取语音地址失败, 请重新尝试';
0025 static String ttsMissingUrlTest(String body) => '语音服务通道发生异常, 请联系开发者获取支持';
0027 // =====
0028 // 业务逻辑与系统状态提示
0029 // =====
0030 static const String ttsRequireBookFirst = '请先导入一部书籍, 再开启语音朗读';
0031 static const String ttsNoContentRequiresBook = '当前没有内容可以朗读, 请先选择一部书籍';
0032 static const String ttsIdleTimeout = '您已经休息很久啦, 为了省电, 语音已自动暂停';
0033 static const String ttsAudioParamFailed = '语音参数调节失败, 请重启功能重试';
0035 // =====
0036 // 本地数据与文件导入
0037 // =====
0038 static const String importFormatFailed = '不支持该书籍的格式, 请确认是否为文本文件';
0039 static const String importReadPathFailed = '无法读取文件路径, 请检查您的存储权限';
0040 static const String testFailedUnresponsive = '连通性测试未通过: 网络连接异常, 服务不可用';
0042 static String importFileTooLarge(int maxMb) =>
0043     '为了保证流畅阅读, 暂不支持体积超过 ${maxMb}MB 的书籍, 建议分卷导入';
0044 }
```

// ===== 文件结束: lib/core/constants/cyber_error_messages.dart =====

// ===== 文件: lib/core/theme/cyber_colors.dart =====

```
0001 import 'package:flutter/material.dart';
0003 class CyberColors {
0004     // 极致的深色背景
0005     static const Color background = Color(0xFF000000); // 真正的高级黑
0007     // 赛博/黑客帝国经典的荧光绿 (主色调)
0008     static const Color neonGreen = Color(0xFF00FF41);
0010     // 赛博朋克必不可少的霓虹粉
0011     static const Color neonPink = Color(0xFFFE019A);
0013     // 霓虹紫色
0014     static const Color neonPurple = Color(0xFF8B5CF6);
0016     // 霓虹青色
0017     static const Color neonCyan = Color(0xFF22D3EE);
0019     // 卡片背景色 (带一点透明度的深灰, 制造玻璃感)
0020     static const Color cardBackground = Color(0xFF13141E);
0021     static const Color surface = Color(0xFF1A1B28);
0023     // 提词器未读文字的暗色
```

```
0024 static const Color textDim = Color(0xFF4A5568);
0026 // 毛玻璃深底色（灵动岛 / 顶部工具栏 / 弹窗共用）
0027 static const Color glassDark = Color(0xD90A0AF);
0029 // 播放按钮渐变粉（CyberPlayerConsole 播放按钮起始色）
0030 static const Color hotPink = Color(0xFFE04899);
0032 // 章节列表 / 弹窗背景
0033 static const Color panelBackground = Color(0xFF0D0E18);
0035 // 白色语义化透明度
0036 static const Color whiteHigh = Color(0xD9FFFFFF); // 85%
0037 static const Color whiteMedium = Color(0x99FFFFFF); // 60%
0038 static const Color whiteDim = Color(0x8AFFFFFF); // 54%
0039 static const Color whiteMuted = Color(0x61FFFFFF); // 38%
0040 static const Color whiteSubtle = Color(0x3DFFFFFF); // 24%
0041 static const Color whiteFaint = Color(0x14FFFFFF); // 8%
0042 static const Color whiteBorder = Color(0x1AFFFFFF); // 10%
0044 // 黑色语义化透明度（阴影 / 遮罩 / overlay）
0045 static const Color blackOverlay = Color(0xB3000000); // 70%
0046 static const Color blackShadow = Color(0x80000000); // 50%
0047 static const Color blackDim = Color(0x8A000000); // 54%
0049 // 柔和的发光阴影色
0050 static const Color glowShadow = Color(0x8800FF41);
0051 static const Color pinkGlow = Color(0x88FE019A);
0053 // 2048 方块配色 - 提取自旧版style.css的渐变色
0054 static const Color tile2Start = Color(0xFF43e97b);
0055 static const Color tile2End = Color(0xFF38f9d7);
0057 static const Color tile4Start = Color(0xFF00f2fe);
0058 static const Color tile4End = Color(0xFF4facfe);
0060 static const Color tile8Start = Color(0xFF4facfe);
0061 static const Color tile8End = Color(0xFF00c6fb);
0063 static const Color tile16Start = Color(0xFF0093e9);
0064 static const Color tile16End = Color(0xFF80d0c7);
0066 static const Color tile32Start = Color(0xFFa18cd1);
0067 static const Color tile32End = Color(0xFFfbc2eb);
0069 static const Color tile64Start = Color(0xFFc471f5);
0070 static const Color tile64End = Color(0xFFfa71cd);
0072 static const Color tile128Start = Color(0xFFfe040fb);
0073 static const Color tile128End = Color(0xFF8e24aa);
0075 static const Color tile256Start = Color(0xFFf857a6);
0076 static const Color tile256End = Color(0xFFff5858);
0078 static const Color tile512Start = Color(0xFFFF7971e);
0079 static const Color tile512End = Color(0xFFFFfd200);
0081 static const Color tile1024Start = Color(0xFFFFf6a00);
0082 static const Color tile1024End = Color(0xFFee0979);
0084 static const Color tile2048Start = Color(0xFFf12711);
0085 static const Color tile2048End = Color(0xFFFF5af19); // 亮红
0086 static const Color tile512 = Color(0xFFFFFD60A); // 金色
0088 // 工具色
0089 static const Color white = Color(0xFFFFFFFF); // 纯白（Canvas 高光专用）
0090 static const Color transparent = Color(0x00000000); // 透明（Material 背景专用）
0091 static const Color tileGold = Color(0xFFFFFD700); // 传奇金色（square_board 溯源）
```

```
0092 static const Color hackerBlue =
0093     Color(0xFF00F3FF); // 骇客蓝 (规范中的 #00f3ff, 吹着天 BGM)
0094 static const Color tileBlue = Color(0xFF3B82F6); // 方块低数粒子色 (tile_widget 溯源)
0095 }
// ===== 文件结束: lib/core/theme/cyber_colors.dart =====

// ===== 文件: lib/core/theme/cyber_dimensions.dart =====
0001 /// 赛博朋克设计系统 - 尺寸规范
0002 /// 统一管理圆角、边框、模糊、间距等数值, 确保视觉一致性
0003 class CyberDimensions {
0004     // ===== 圆角系统 =====
0005     // 5 级圆角, 从大到小递减
0007     /// 超大圆角 - 用于大型容器 (棋盘外框)
0008     static const double radiusXL = 32.0;
0010     /// 大圆角 - 用于中型容器 (控制台、工具栏、弹窗)
0011     static const double radiusL = 24.0;
0013     /// 中圆角 - 用于卡片、格子
0014     static const double radiusM = 16.0;
0016     /// 小圆角 - 用于按钮、小卡片
0017     static const double radiusS = 12.0;
0019     /// 超小圆角 - 用于细节 (进度条、波形条)
0020     static const double radiusXS = 2.0;
0022     // ===== 边框系统 =====
0023     // 3 级边框宽度
0025     /// 粗边框 - 用于强调 (选中状态、主要边框)
0026     static const double borderThick = 1.5;
0028     /// 常规边框 - 用于普通容器
0029     static const double borderNormal = 1.0;
0031     /// 细边框 - 用于细微装饰 (背景网格)
0032     static const double borderThin = 0.5;
0034     // ===== 毛玻璃模糊系统 =====
0035     // 3 级模糊强度
0037     /// 强模糊 - 用于主要容器 (棋盘、卡片)
0038     static const double blurStrong = 20.0;
0040     /// 中模糊 - 用于工具栏、控制台
0041     static const double blurMedium = 15.0;
0043     /// 轻模糊 - 用于遮罩、头部
0044     static const double blurLight = 10.0;
0046     // ===== 间距系统 =====
0047     // 标准间距, 8px 倍数
0049     static const double spacingXXS = 2.0;
0050     static const double spacingXS = 4.0;
0051     static const double spacingS = 8.0;
0052     static const double spacingMS = 12.0;
0053     static const double spacingM = 16.0;
0054     static const double spacingML = 20.0;
0055     static const double spacingL = 24.0;
0056     static const double spacingXL = 32.0;
0058     // ===== 头部 / 工具栏 =====
0060     /// 标准页面头部高度
```

```
0061 static const double headerHeight = 56.0;
0063 /// 标准电传屏高度
0064 static const double teleprompterHeight = 46.0;
0066 /// 标准电传屏遮罩宽度
0067 static const double teleprompterMaskWidth = 40.0;
0069 /// 标准仪表盘吉祥物宽度
0070 static const double dashboardMascotWidth = 68.0;
0072 /// 标准仪表盘吉祥物高度
0073 static const double dashboardMascotHeight = 84.0;
0075 /// 标准仪表盘缓冲区高度
0076 static const double dashboardBoardBuffer = 76.0;
0078 /// 标准仪表盘状态卡片最小高度
0079 static const double dashboardStatusCardMinHeight = 85.0;
0081 // ===== 动效时长 =====
0083 /// 通用过渡动画时长（Modal、开关、按钮状态切换）
0084 static const Duration animNormal = Duration(milliseconds: 300);
0086 /// 快速反馈动画时长（弹窗退场、选项切换）
0087 static const Duration animFast = Duration(milliseconds: 250);
0089 /// 超快反馈动画时长（棋盘倾斜、元素位移）
0090 static const Duration animXFast = Duration(milliseconds: 150);
0092 /// 瞬间反馈动画时长（眼球追踪、合并弹出）
0093 static const Duration animInstant = Duration(milliseconds: 120);
0095 /// 中等动画时长（合并粒子、导入按钮）
0096 static const Duration animMedium = Duration(milliseconds: 400);
0098 /// 消除动画时长（棋盘方块消除三段式）
0099 static const Duration animEliminate = Duration(milliseconds: 450);
0101 /// 慢动画时长（棋盘重置、身体跳跃）
0102 static const Duration animSlow = Duration(milliseconds: 600);
0104 /// Toast 显示持续时长
0105 static const Duration toastDuration = Duration(seconds: 2, milliseconds: 500);
0107 // ===== 动效（视觉） =====
0109 /// 提词器扫光条宽度
0110 static const double shimmerWidth = 80.0;
0112 /// 发光阴影模糊半径（霓虹光晕）
0113 static const double glowBlurRadius = 8.0;
0115 /// 发光阴影扩散半径
0116 static const double glowSpreadRadius = 1.0;
0118 // ===== 列表 / 网格 =====
0120 /// 章节列表 item 固定高度（复用 headerHeight）
0121 static const double chapterItemHeight = headerHeight;
0123 /// 状态圆点尺寸（已读/未读指示器）
0124 static const double statusDotSize = 6.0;
0126 // ===== 图标尺寸 =====
0128 /// 超小图标（箭头、指示器）
0129 static const double iconXS = 16.0;
0131 /// 小图标（音量、状态）
0132 static const double iconS = 18.0;
0134 /// 中图标（操作按钮、加载器）
0135 static const double iconM = 20.0;
0137 /// 大图标（对话框标题图标）
```

```
0138   static const double iconL = 28.0;
0139 }
// ===== 文件结束: lib/core/theme/cyber_dimensions.dart =====

// ===== 文件: lib/core/theme/cyber_shadows.dart =====
0001 import 'package:flutter/material.dart';
0002 import 'cyber_colors.dart';
0004 /// 赛博朋克设计系统 - 阴影规范
0005 /// 统一管理阴影效果, 确保视觉层次一致
0006 class CyberShadows {
0007   // ===== 标准阴影系统 =====
0009   /// 悬浮阴影 - 用于大型容器 (棋盘、卡片)
0010   /// 强烈的悬浮感, 适合主要内容区域
0011   static const List<BoxShadow> elevated = [
0012     BoxShadow(
0013       color: CyberColors.blackShadow,
0014       blurRadius: 30,
0015       offset: Offset(0, 10),
0016     ),
0017   ];
0019   /// 浮动阴影 - 用于中型组件 (书架卡片、导入按钮)
0020   /// 轻微悬浮感, 适合次要内容
0021   static const List<BoxShadow> floating = [
0022     BoxShadow(
0023       color: CyberColors.blackShadow,
0024       blurRadius: 16,
0025       offset: Offset(0, 6),
0026     ),
0027   ];
0029   /// 贴近阴影 - 用于小型组件 (播放按钮、小卡片)
0030   /// 贴近表面, 适合细节装饰
0031   static const List<BoxShadow> subtle = [
0032     BoxShadow(
0033       color: CyberColors.blackShadow,
0034       blurRadius: 10,
0035       offset: Offset(0, 4),
0036     ),
0037   ];
0039   // ===== 霓虹光晕阴影 =====
0041   /// 创建霓虹光晕效果
0042   /// 用于方块、按钮等需要发光效果的元素
0043   static List<BoxShadow> neonGlow({
0044     required Color color,
0045     double intensity = 1.0,
0046   }) {
0047     return [
0048       BoxShadow(
0049         color: color.withValues(alpha: 0.6 * intensity),
0050         blurRadius: 12 * intensity,
0051         spreadRadius: 2 * intensity,
```

```

0052     ),
0053     BoxShadow(
0054       color: color.withValues(alpha: 0.3 * intensity),
0055       blurRadius: 18 * intensity,
0056       spreadRadius: 0,
0057     ),
0058   ];
0059 }
0060 }

// ===== 文件结束: lib/core/theme/cyber_shadows.dart =====

// ===== 文件: lib/core/theme/cyber_text_styles.dart =====
0001 import 'package:flutter/material.dart';
0002 import 'cyber_colors.dart';
0003 class CyberTextStyles {
0004   static const String monoFont = 'JetBrains Mono';
0005   // 提词器正在阅读的【高亮发光文字】
0006   static const TextStyle teleprompterActive = TextStyle(
0007     color: CyberColors.neonGreen,
0008     fontSize: 24,
0009     fontWeight: FontWeight.bold,
0010     letterSpacing: 2.0, // 字间距拉开, 更有极客敲代码的呼吸感
0011     shadows: [
0012       Shadow(
0013         blurRadius: 10.0, // 发光半径
0014         color: CyberColors.glowShadow,
0015         offset: Offset(0, 0),
0016       ),
0017     ],
0018   );
0019   // 提词器下方还没读到的【暗色背景文字】
0020   static const TextStyle teleprompterDim = TextStyle(
0021     color: CyberColors.textDim,
0022     fontSize: 24,
0023     fontWeight: FontWeight.normal,
0024     letterSpacing: 2.0,
0025   );
0026   // 2048 棋盘上的数字字体
0027   static const TextStyle gameGridNumber = TextStyle(
0028     color: CyberColors.background,
0029     fontSize: 28,
0030     fontWeight: FontWeight.w900,
0031   );
0032   // ===== 通用 UI 文本样式 =====
0033   /// 页面头部标题 (18px, bold, letterSpacing: 2)
0034   /// 使用时通过 copyWith(color: ...) 设置具体颜色
0035   static const TextStyle screenTitle = TextStyle(
0036     fontSize: 18,
0037     fontWeight: FontWeight.bold,
0038     letterSpacing: 2,

```

```
0044 );
0046 /// 分区标题 (12px, bold, letterSpacing: 1.5, neonGreen)
0047 static const TextStyle sectionLabel = TextStyle(
0048   color: CyberColors.neonGreen,
0049   fontSize: 12,
0050   fontWeight: FontWeight.bold,
0051   letterSpacing: 1.5,
0052 );
0054 /// 列表项主标题 (14px, whiteHigh)
0055 static const TextStyle tileTitle = TextStyle(
0056   color: CyberColors.whiteHigh,
0057   fontSize: 14,
0058 );
0060 /// 列表项副标题 (12px, whiteMuted)
0061 static const TextStyle tileSubtitle = TextStyle(
0062   color: CyberColors.whiteMuted,
0063   fontSize: 12,
0064 );
0066 /// 标签文字 (14px, whiteDim)
0067 static const TextStyle labelMedium = TextStyle(
0068   color: CyberColors.whiteDim,
0069   fontSize: 14,
0070 );
0072 /// 小号正文 (13px, whiteDim) —— 下拉菜单、标签芯片等
0073 static const TextStyle bodySmall = TextStyle(
0074   color: CyberColors.whiteDim,
0075   fontSize: 13,
0076 );
0078 /// 小号正文加粗 (13px, w600, whiteHigh)
0079 static const TextStyle bodySmallBold = TextStyle(
0080   color: CyberColors.whiteHigh,
0081   fontSize: 13,
0082   fontWeight: FontWeight.w600,
0083 );
0085 static const TextStyle overlineTiny = TextStyle(
0086   color: CyberColors.whiteDim,
0087   fontSize: 9,
0088   fontWeight: FontWeight.w700,
0089   letterSpacing: 0.5,
0090 );
0092 static const TextStyle segmentLabel = TextStyle(
0093   color: CyberColors.whiteHigh,
0094   fontSize: 14,
0095   fontWeight: FontWeight.w600,
0096   letterSpacing: 0.3,
0097 );
0099 static const TextStyle teleprompterInlineRead = TextStyle(
0100   color: CyberColors.neonCyan,
0101   fontSize: 18,
0102   fontWeight: FontWeight.w800,
```

```
0103     letterSpacing: 0.5,
0104     height: 1.0,
0105 );
0107 static const TextStyle teleprompterInlineUnread = TextStyle(
0108     color: CyberColors.whiteMuted,
0109     fontSize: 18,
0110     fontWeight: FontWeight.w500,
0111     letterSpacing: 0.5,
0112     height: 1.0,
0113 );
0115 static const TextStyle teleprompterError = TextStyle(
0116     color: CyberColors.whiteHigh,
0117     fontSize: 12,
0118     fontWeight: FontWeight.w700,
0119 );
0121 /// 提词器浮层小号错误标题 (10px bold)
0122 static const TextStyle teleprompterErrorTitle = TextStyle(
0123     fontSize: 10,
0124     fontWeight: FontWeight.bold,
0125 );
0127 /// 提词器浮层小号提示文字 (9px)
0128 static const TextStyle teleprompterErrorHint = TextStyle(
0129     fontSize: 9,
0130 );
0132 static const TextStyle teleprompterPlaceholder = TextStyle(
0133     color: CyberColors.whiteMuted,
0134     fontSize: 14,
0135     fontWeight: FontWeight.w600,
0136 );
0138 static const TextStyle dashboardCounter = TextStyle(
0139     color: CyberColors.neonCyan,
0140     fontSize: 22,
0141     fontFamily: monoFont,
0142     fontWeight: FontWeight.w900,
0143 );
0145 static const TextStyle dashboardSeparator = TextStyle(
0146     color: CyberColors.whiteSubtle,
0147     fontSize: 16,
0148     fontFamily: monoFont,
0149     fontWeight: FontWeight.w300,
0150 );
0152 static const TextStyle captionBold = TextStyle(
0153     color: CyberColors.whiteDim,
0154     fontSize: 12,
0155     fontWeight: FontWeight.bold,
0156 );
0158 static const TextStyle captionTight = TextStyle(
0159     color: CyberColors.whiteDim,
0160     fontSize: 12,
0161     height: 1.3,
```



```

0162 );
0164 static const TextStyle captionComfortable = TextStyle(
0165   color: CyberColors.whiteDim,
0166   fontSize: 12,
0167   height: 1.4,
0168 );
0170 static const TextStyle captionHint = TextStyle(
0171   color: CyberColors.whiteDim,
0172   fontSize: 11,
0173   height: 1.5,
0174 );
0176 /// 提示/说明文字 (12px, whiteDim)
0177 static const TextStyle caption = TextStyle(
0178   color: CyberColors.whiteDim,
0179   fontSize: 12,
0180 );
0182 /// 对话框标题 (18px, bold)
0183 /// 使用时通过 copyWith(color: ...) 设置具体颜色
0184 static const TextStyle dialogTitle = TextStyle(
0185   fontSize: 18,
0186   fontWeight: FontWeight.bold,
0187 );
0189 /// 按钮文字 (14px, bold)
0190 static const TextStyle buttonLabel = TextStyle(
0191   fontSize: 14,
0192   fontWeight: FontWeight.bold,
0193 );
0194 }

// ===== 文件结束: lib/core/theme/cyber_text_styles.dart =====

// ===== 文件: lib/core/database/i_storage_service.dart =====
0001 /// 本地持久化服务的统一接口定义 (依赖倒置原则)
0002 ///
0003 /// 所有业务逻辑层应依赖本接口, 而非直接引用 [StorageService] 静态类。
0004 /// 这样可在测试中注入 Mock 实现, 在生产环境中注入 [LocalStorageService],
0005 /// 实现业务逻辑与具体存储实现的完全解耦。
0006 ///
0007 /// ## 设计约束
0008 /// - 本接口位于 `core/database/`, 仅允许被 `features/*/providers/` 消费。
0009 /// - 严禁在 `presentation/` 层直接调用任何存储接口。
0010 /// - 所有写操作均为 async`Future<void>`, 读操作可同步或异步。
0011 abstract interface class IStorageService {
0012   // —— 初始化 ——
0014   /// 初始化底层存储引擎 (应在 [runApp] 前调用一次)
0015   Future<void> init();
0017   // —— 游戏状态 ——
0019   /// 保存完整的 2048 游戏快照
0020   ///
0021   /// - [board]: 4×4 棋盘 (`null` 表示空格, 非 `null` 为 `{id, value}`)
0022   /// - [score]: 当前全盘总分 (全盘求和法, 非累加法)

```

```
0023  /// - [combo]: 当前连击数
0024  /// - [bestScore]: 历史最佳得分
0025  /// - [maxCombo]: 历史最大连击
0026  /// - [novelIndex]: 当前关联的小说索引
0027  /// - [currentNovelId]: 当前关联的小说 ID (可为 null)
0028  Future<void> saveGameState({
0029      required List<List<Map<String, dynamic?>>> board,
0030      required int score,
0031      required int combo,
0032      required int bestScore,
0033      required int maxCombo,
0034      required int novelIndex,
0035      String? currentNovelId,
0036  });
0038  /// 读取游戏快照, 返回 `null` 表示无存档或存档损坏
0039  Map<String, dynamic>? loadGameState();
0041  /// 读取历史最佳得分 (无存档时返回 0)
0042  int loadBestScore();
0044  /// 读取历史最大连击数 (无存档时返回 0)
0045  int loadMaxCombo();
0047  // —— 书架元数据 ——
0049  /// 保存书架元数据列表 (全量覆盖写入)
0050  ///
0051  /// 每个元素为书籍元数据 Map, 至少包含 `id`、`title`、`total` 字段。
0052  Future<void> saveBookshelf(List<Map<String, dynamic>> shelf);
0054  /// 读取书架元数据列表 (损坏或空时返回空列表)
0055  List<Map<String, dynamic>> loadBookshelf();
0057  // —— 阅读进度 ——
0059  /// 更新指定书籍的阅读进度
0060  ///
0061  /// - [bookId]: 书籍唯一标识
0062  /// - [cursor]: 当前行号 (0-based)
0063  /// - [total]: 总行数 (≤ 0 时忽略, 防止除零)
0064  Future<void> updateReadingRecord(String bookId, int cursor, int total);
0066  /// 读取指定书籍的阅读进度
0067  ///
0068  /// 返回 `{cursor, total, percent}`, 无记录时返回默认零进度。
0069  Map<String, dynamic> getReadingRecord(String bookId);
0071  /// 删除指定书籍的阅读进度记录
0072  Future<void> deleteReadingRecord(String bookId);
0074  // —— 当前小说状态 ——
0076  /// 设置当前激活的小说 ID (传入 `null` 时清除)
0077  Future<void> setCurrentNovelId(String? id);
0079  /// 读取当前激活的小说 ID (未设置时返回 `null`)
0080  String? getCurrentNovelId();
0082  /// 读取当前激活的小说在书架中的索引 (未设置时返回 0)
0083  int getCurrentNovelIndex();
0085  /// 设置当前激活的小说在书架中的索引
0086  Future<void> setCurrentNovelIndex(int index);
0088  // —— 书籍正文内容 ——
```

```
0090  /// 将书籍正文按行存储为 JSON 文件（大文件，存入 Documents 目录）
0091  ///
0092  /// - [bookId]: 书籍唯一标识（作为文件名）
0093  /// - [lines]: 全文按行分割后的字符串列表
0094  /// - [chapters]: 章节目录，每项至少含 `title`、`lineIndex` 字段
0095  Future<void> saveBookContent(
0096      String bookId, {
0097      required List<String> lines,
0098      required List<Map<String, dynamic>> chapters,
0099  });
0101  /// 读取书籍正文内容
0102  ///
0103  /// 返回 `{lines: List<String>, chapters: List<Map>}`，
0104  /// 文件不存在或 JSON 损坏时返回 `null`。
0105  Future<Map<String, dynamic>?> loadBookContent(String bookId);
0107  /// 删除指定书籍的正文文件（文件不存在时静默忽略）
0108  Future<void> deleteBookContent(String bookId);
0110  // —— 全局设置 ——
0112  /// 读取音效开关（默认 `true`）
0113  bool getSettingSound();
0115  /// 设置音效开关并持久化
0116  Future<void> setSettingSound(bool v);
0118  /// 读取 TTS 朗读开关（默认 `true`）
0119  bool getSettingStoryTts();
0121  /// 设置 TTS 朗读开关并持久化
0122  Future<void> setSettingStoryTts(bool v);
0124  /// 读取 TTS 音色 ID（默认 `zh-CN-XiaoxiaoNeural`）
0125  String getSettingVoice();
0127  /// 设置 TTS 音色 ID 并持久化
0128  Future<void> setSettingVoice(String v);
0130  /// 读取空闲超时（分钟，默认 1）
0131  int getSettingIdleTimeout();
0133  /// 设置空闲超时并持久化
0134  Future<void> setSettingIdleTimeout(int v);
0136  /// 读取 TTS 播放倍速（默认 `1.0`）
0137  double getSettingTtsRate();
0139  /// 设置 TTS 播放倍速并持久化
0140  Future<void> setSettingTtsRate(double v);
0142  /// 读取环境音量（默认 `0.5`，范围 `0.0` - `1.0`）
0143  double getSettingAmbientVol();
0145  /// 设置环境音量并持久化
0146  Future<void> setSettingAmbientVol(double v);
0148  /// 读取环境背景音乐开关（默认 `true`）
0149  bool getSettingAmbientEnabled();
0151  /// 设置环境背景音乐开关并持久化
0152  Future<void> setSettingAmbientEnabled(bool v);
0154  // —— 隐私协议 ——
0156  /// 读取用户是否已同意隐私协议（默认 `false`）
0157  bool hasAgreedPrivacy();
0159  /// 设置隐私协议同意状态并持久化
```

```

0160 Future<void> setHasAgreedPrivacy(bool v);
0162 // —— 粘性位：用户是否曾主动选择过书籍 ——
0164 /// 读取用户是否曾主动选择过书籍（默认 `false`）
0165 ///
0166 /// 用于区分“从未有过书籍的新用户”与“删掉所有书籍的老用户”，
0167 /// 仅前者在书架为空时触发内置默认书籍自动注入。
0168 bool hasSelectedBook();
0170 /// 设置粘性位并持久化
0171 Future<void> setHasSelectedBook(bool v);
0173 // —— 分章文本缓存（文件路径，非 SharedPreferences） ——
0175 /// 将指定书籍的某章节纯文本写入本地文件
0176 ///
0177 /// 文件路径：`{documentsDir}/books/chapters/{bookId}/{chapterIndex:03d}.txt`
0178 Future<void> saveChapterCache(String bookId, int chapterIndex, String text);
0180 /// 读取指定书籍章节的本地缓存文本
0181 ///
0182 /// 文件不存在时返回 `null`。
0183 Future<String?> loadChapterCache(String bookId, int chapterIndex);
0185 /// 删除指定书籍所有已缓存章节文件
0186 Future<void> clearChapterCache(String bookId);
0188 /// LRU 清理：仅保留 `currentChapterIndex ± keepAround` 范围内的章节缓存
0189 ///
0190 /// 用于防止长时间阅读后缓存文件堆积。
0191 Future<void> pruneChapterCache(
0192     String bookId,
0193     int currentChapterIndex, {
0194     int keepAround = 3,
0195 });
0197 // —— 书目录缓存（文件路径） ——
0199 /// 将书目录（章节标题 + 索引列表）序列化写入本地文件
0200 ///
0201 /// 文件路径：`{documentsDir}/books/catalogs/{bookId}_catalog.json`
0202 Future<void> saveBookCatalog(
0203     String bookId,
0204     List<Map<String, dynamic>> chapters,
0205 );
0207 /// 读取书目录缓存
0208 ///
0209 /// 文件不存在或 JSON 损坏时返回 `null`。
0210 Future<List<Map<String, dynamic>>> loadBookCatalog(String bookId);
0211 }

// ===== 文件结束：lib/core/database/i_storage_service.dart =====

// ===== 文件：lib/core/database/local_storage_service.dart =====
0001 import 'package:yueyou/core/database/i_storage_service.dart';
0002 import 'package:yueyou/core/database/storage_service.dart';
0004 /// [IStorageService] 的本地持久化实现（生产环境注入版本）
0005 ///
0006 /// 内部将所有调用委托给 [StorageService] 静态方法，保持与现有
0007 /// SharedPreferences + path_provider 存储行为 100% 一致。

```

```

0008 ///
0009 /// ## 用法
0010 /// ```dart
0011 /// // 在 main.dart 中完成初始化
0012 /// final storage = LocalStorageService();
0013 /// await storage.init();
0014 /// runApp(MultiProvider(providers: [
0015 ///   Provider<IStorageService>.value(value: storage),
0016 ///   // ... 其他 Provider
0017 /// ]));
0018 /// ```
0019 ///
0020 /// ## 测试替换
0021 /// 在测试中注入 [MockStorageService] (实现 [IStorageService]) ,
0022 /// 无需依赖 SharedPreferences, 实现完全隔离。
0023 class LocalStorageService implements IStorageService {
0024   // —— 初始化 ——
0025
0026   @override
0027   Future<void> init() => StorageService.init();
0028
0029   // —— 游戏状态 ——
0030
0031   @override
0032   Future<void> saveGameState({
0033     required List<List<Map<String, dynamic?>>> board,
0034     required int score,
0035     required int combo,
0036     required int bestScore,
0037     required int maxCombo,
0038     required int novelIndex,
0039     String? currentNovelId,
0040   }) =>
0041     StorageService.saveGameState(
0042       board: board,
0043       score: score,
0044       combo: combo,
0045       bestScore: bestScore,
0046       maxCombo: maxCombo,
0047       novelIndex: novelIndex,
0048       currentNovelId: currentNovelId,
0049     );
0050
0051   @override
0052   Map<String, dynamic?> loadGameState() => StorageService.loadGameState();
0053
0054   @override
0055   int loadBestScore() => StorageService.loadBestScore();
0056
0057   @override
0058   int loadMaxCombo() => StorageService.loadMaxCombo();
0059
0060   // —— 书架元数据 ——
0061
0062   @override
0063   Future<void> saveBookshelf(List<Map<String, dynamic>> shelf) =>
0064     StorageService.saveBookshelf(shelf);
0065
0066   @override

```

```
0067 List<Map<String, dynamic>> loadBookshelf() => StorageService.loadBookshelf();
0069 // —— 阅读进度 ——
0071 @override
0072 Future<void> updateReadingRecord(String bookId, int cursor, int total) =>
0073     StorageService.updateReadingRecord(bookId, cursor, total);
0075 @override
0076 Map<String, dynamic> getReadingRecord(String bookId) =>
0077     StorageService.getReadingRecord(bookId);
0079 @override
0080 Future<void> deleteReadingRecord(String bookId) =>
0081     StorageService.deleteReadingRecord(bookId);
0083 // —— 当前小说状态 ——
0085 @override
0086 Future<void> setCurrentNovelId(String? id) =>
0087     StorageService.setCurrentNovelId(id);
0089 @override
0090 String? getCurrentNovelId() => StorageService.getCurrentNovelId();
0092 @override
0093 int getCurrentNovelIndex() => StorageService.getCurrentNovelIndex();
0095 @override
0096 Future<void> setCurrentNovelIndex(int index) =>
0097     StorageService.setCurrentNovelIndex(index);
0099 // —— 书籍正文内容 ——
0101 @override
0102 Future<void> saveBookContent(
0103     String bookId, {
0104     required List<String> lines,
0105     required List<Map<String, dynamic>> chapters,
0106 }) =>
0107     StorageService.saveBookContent(bookId, lines: lines, chapters: chapters);
0109 @override
0110 Future<Map<String, dynamic>?> loadBookContent(String bookId) =>
0111     StorageService.loadBookContent(bookId);
0113 @override
0114 Future<void> deleteBookContent(String bookId) =>
0115     StorageService.deleteBookContent(bookId);
0117 // —— 全局设置 ——
0119 @override
0120 bool getSettingSound() => StorageService.getSettingSound();
0122 @override
0123 Future<void> setSettingSound(bool v) => StorageService.setSettingSound(v);
0125 @override
0126 bool getSettingStoryTts() => StorageService.getSettingStoryTts();
0128 @override
0129 Future<void> setSettingStoryTts(bool v) =>
0130     StorageService.setSettingStoryTts(v);
0132 @override
0133 String getSettingVoice() => StorageService.getSettingVoice();
0135 @override
0136 Future<void> setSettingVoice(String v) => StorageService.setSettingVoice(v);
```

```
0138  @override
0139  int getSettingIdleTimeout() => StorageService.getSettingIdleTimeout();
0141  @override
0142  Future<void> setSettingIdleTimeout(int v) =>
0143      StorageService.setSettingIdleTimeout(v);
0145  @override
0146  double getSettingTtsRate() => StorageService.getSettingTtsRate();
0148  @override
0149  Future<void> setSettingTtsRate(double v) =>
0150      StorageService.setSettingTtsRate(v);
0152  @override
0153  double getSettingAmbientVol() => StorageService.getSettingAmbientVol();
0155  @override
0156  Future<void> setSettingAmbientVol(double v) =>
0157      StorageService.setSettingAmbientVol(v);
0159  @override
0160  bool getSettingAmbientEnabled() => StorageService.getSettingAmbientEnabled();
0162  @override
0163  Future<void> setSettingAmbientEnabled(bool v) =>
0164      StorageService.setSettingAmbientEnabled(v);
0166  // —— 隐私协议 ——
0168  @override
0169  bool hasAgreedPrivacy() => StorageService.hasAgreedPrivacy();
0171  @override
0172  Future<void> setHasAgreedPrivacy(bool v) =>
0173      StorageService.setHasAgreedPrivacy(v);
0175  // —— 粘性位：用户是否曾主动选择过书籍 ——
0177  @override
0178  bool hasSelectedBook() => StorageService.hasSelectedBook();
0180  @override
0181  Future<void> setHasSelectedBook(bool v) =>
0182      StorageService.setHasSelectedBook(v);
0184  // —— 分章文本缓存 ——
0186  @override
0187  Future<void> saveChapterCache(String bookId, int chapterIndex, String text) =>
0188      StorageService.saveChapterCache(bookId, chapterIndex, text);
0190  @override
0191  Future<String?> loadChapterCache(String bookId, int chapterIndex) =>
0192      StorageService.loadChapterCache(bookId, chapterIndex);
0194  @override
0195  Future<void> clearChapterCache(String bookId) =>
0196      StorageService.clearChapterCache(bookId);
0198  @override
0199  Future<void> pruneChapterCache(
0200      String bookId,
0201      int currentChapterIndex, {
0202      int keepAround = 3,
0203  }) =>
0204      StorageService.pruneChapterCache(
0205      bookId,
```



```
// ===== 后 30 页连续源码 =====
// ===== 文件: lib/features/settings/presentation/widgets/privacy_agreement_modal.dart =====
0001 import 'package:flutter/material.dart';
0002 import 'package:flutter/services.dart';
0003 import 'package:url_launcher/url_launcher.dart';
0004 import 'package:yueyou/core/config/app_info_config.dart';
0005 import 'package:yueyou/core/theme/cyber_colors.dart';
0006 import 'package:yueyou/core/theme/cyber_dimensions.dart';
0007 import 'package:yueyou/core/theme/cyber_text_styles.dart';
0008 import 'package:yueyou/shared/widgets/cyber_modal.dart';
0010 /// 赛博朋克风格隐私协议弹窗
0011 /// - barrierDismissible: false, 强制用户作出选择
0012 /// - 返回 true 表示同意, 用户拒绝时直接退出应用
0013 Future<bool> showPrivacyAgreementModal(BuildContext context) async {
0014   final result = await showCyberModal<bool>(
0015     context: context,
0016     barrierDismissible: false,
0017     child: const _PrivacyAgreementContent(),
0018   );
0019   return result ?? false;
0020 }
0022 class _PrivacyAgreementContent extends StatelessWidget {
0023   const _PrivacyAgreementContent();
0025   @override
0026   Widget build(BuildContext context) {
0027     return Padding(
0028       padding: const EdgeInsets.all(CyberDimensions.spacingL),
0029       child: Column(
0030         mainAxisAlignment: MainAxisAlignment.min,
0031         children: [
0032           // —— 图标 + 标题 ——
0033           const Icon(
0034             Icons.security_rounded,
0035             color: CyberColors.neonCyan,
0036             size: CyberDimensions.iconL,
0037           ),
0038           const SizedBox(height: CyberDimensions.spacingS),
0039           Text(
0040             '阅游 · 隐私政策',
0041             style: CyberTextStyles.dialogTitle.copyWith(
0042               color: CyberColors.neonCyan,
0043             ),
0044           ),
0045           const SizedBox(height: CyberDimensions.spacingXS),
0046           Text(
0047             '神经接驳协议 · NEURAL LINK v1.0',
0048             style: CyberTextStyles.caption.copyWith(
0049               color: CyberColors.whiteMuted,
0050               letterSpacing: 1.5,
0051             ),

```

```

0052     ),
0053     const SizedBox(height: CyberDimensions.spacingM),
0054     // —— 协议正文（可滚动） ——
0055     Container(
0056       constraints: BoxConstraints(
0057         maxHeight: MediaQuery.sizeOf(context).height * 0.4,
0058       ),
0059       decoration: BoxDecoration(
0060         color: CyberColors.whiteFaint,
0061         borderRadius: BorderRadius.circular(CyberDimensions.radiusS),
0062         border: Border.all(
0063           color: CyberColors.neonCyan.withValues(alpha: 0.2),
0064           width: CyberDimensions.borderThin,
0065         ),
0066       ),
0067     ),
0068     child: const SingleChildScrollView(
0069       padding: EdgeInsets.all(CyberDimensions.spacingM),
0070       child: Column(
0071         crossAxisAlignment: CrossAxisAlignment.start,
0072         children: [
0073           _PolicySection(
0074             icon: '',
0075             title: '数据存储',
0076             body: '所有阅读进度、游戏数据及用户设置仅存储于您的本地设备。'
0077               '我们不会将您的个人数据上传至任何远端服务器。',
0078           ),
0079           SizedBox(height: CyberDimensions.spacingM),
0080           _PolicySection(
0081             icon: '',
0082             title: '云端 TTS 合成',
0083             body: '当您启用听书功能时，当前朗读的文本段落将通过加密信道发送至'
0084               '云端 TTS 服务以合成音频。除朗读文本外，不传输任何其他信息。',
0085           ),
0086           SizedBox(height: CyberDimensions.spacingM),
0087           _PolicySection(
0088             icon: '',
0089             title: '存储权限',
0090             body: '导入数据芯片（TXT 文件）时，本应用需要访问您设备的本地存储。'
0091               '此权限仅用于读取您主动选择的文件，不会扫描其他目录。',
0092           ),
0093           SizedBox(height: CyberDimensions.spacingM),
0094           _PolicySection(
0095             icon: '',
0096             title: '隐私承诺',
0097             body: '- 不采集设备标识符或位置信息\n'
0098               '- 不追踪用户行为或使用习惯\n'
0099               '- 不向任何第三方共享数据\n'
0100               '- 符合《个人信息保护法》及 GDPR',
0101           ),
0102           SizedBox(height: CyberDimensions.spacingM),

```



```

0103         _PolicySection(
0104             icon: '',
0105             title: '开发者信息',
0106             body: '运营主体: ${AppInfoConfig.developerName}\n'
0107                 '联系邮箱: ${AppInfoConfig.contactEmail}\n'
0108                 '隐私问题反馈、数据删除请求请发送邮件至上述邮箱。',
0109         ),
0110     ],
0111 ),
0112 ),
0113 ),
0114 const SizedBox(height: CyberDimensions.spacingM),
0115 // —— 提示 + 隐私政策外链 ——
0116 Wrap(
0117     alignment: WrapAlignment.center,
0118     crossAxisAlignment: WrapCrossAlignment.center,
0119     children: [
0120         Text(
0121             '同意后, 您可在设置页面随时重新查看本协议 ',
0122             textAlign: TextAlign.center,
0123             style: CyberTextStyles.caption.copyWith(
0124                 color: CyberColors.whiteMuted,
0125             ),
0126         ),
0127         GestureDetector(
0128             onTap: () => launchUrl(
0129                 Uri.parse(AppInfoConfig.privacyPolicyUrl),
0130                 mode: LaunchMode.externalApplication,
0131             ),
0132             child: Text(
0133                 '阅读完整版《阅游隐私政策》',
0134                 style: CyberTextStyles.caption.copyWith(
0135                     color: CyberColors.neonCyan,
0136                     decoration: TextDecoration.underline,
0137                     decorationColor: CyberColors.neonCyan,
0138                 ),
0139             ),
0140         ),
0141     ],
0142 ),
0143 ),
0144 const SizedBox(height: CyberDimensions.spacingL),
0145 // —— 按钮行 ——
0146 Row(
0147     children: [
0148         // 不同意并退出 (危险操作, 用粉色/红色边框区分)
0149         Expanded(
0150             child: _DeclineButton(
0151                 onTap: () {
0152                     Navigator.of(context).pop(false);
0153                     SystemNavigator.pop();
0154                 }

```

```

0155         },
0156     ),
0157 ),
0158     const SizedBox(width: CyberDimensions.spacingMS),
0159     // 同意（主按钮）
0160     Expanded(
0161         child: _AgreeButton(
0162             onTap: () => Navigator.of(context).pop(true),
0163         ),
0164     ),
0165 ],
0166 ),
0167 ],
0168 ),
0169 );
0170 }
0171 }
0173 // —— 协议条款单节 ——
0174 class _PolicySection extends StatelessWidget {
0175     final String icon;
0176     final String title;
0177     final String body;
0178     const _PolicySection({
0179         required this.icon,
0180         required this.title,
0181         required this.body,
0182     });
0183     @override
0184     Widget build(BuildContext context) {
0185         return Column(
0186             crossAxisAlignment: CrossAxisAlignment.start,
0187             children: [
0188                 Row(
0189                     children: [
0190                         Text(
0191                             icon,
0192                             style: CyberTextStyles.bodySmall.copyWith(
0193                                 fontSize: 14,
0194                             ),
0195                         ),
0196                         const SizedBox(width: CyberDimensions.spacingXS),
0197                         Text(
0198                             title,
0199                             style: CyberTextStyles.bodySmallBold.copyWith(
0200                                 color: CyberColors.neonCyan,
0201                             ),
0202                         ),
0203                     ],
0204                 ),
0205                 const SizedBox(height: CyberDimensions.spacingXS),

```

```

0208     Text(
0209         body,
0210         style: CyberTextStyles.captionComfortable.copyWith(
0211             color: CyberColors.whiteMedium,
0212         ),
0213     ),
0214 ],
0215 );
0216 }
0217 }

0219 // —— 拒绝按钮（霓虹粉边框，危险语义） ——
0220 class _DeclineButton extends StatelessWidget {
0221     final VoidCallback onTap;
0222     const _DeclineButton({required this.onTap});
0223
0224     @override
0225     Widget build(BuildContext context) {
0226         return GestureDetector(
0227             onTap: onTap,
0228             child: Container(
0229                 padding:
0230                     const EdgeInsets.symmetric(vertical: CyberDimensions.spacingMS),
0231                 decoration: BoxDecoration(
0232                     color: CyberColors.neonPink.withValues(alpha: 0.08),
0233                     borderRadius: BorderRadius.circular(CyberDimensions.radiusS),
0234                     border: Border.all(
0235                         color: CyberColors.neonPink.withValues(alpha: 0.5),
0236                         width: CyberDimensions.borderNormal,
0237                     ),
0238                 ),
0239                 child: Center(
0240                     child: Text(
0241                         '不同意并退出',
0242                         style: CyberTextStyles.buttonLabel.copyWith(
0243                             color: CyberColors.neonPink,
0244                         ),
0245                     ),
0246                 ),
0247             ),
0248         );
0249     }
0250 }

0252 // —— 同意按钮（霓虹青渐变，主语义） ——
0253 class _AgreeButton extends StatelessWidget {
0254     final VoidCallback onTap;
0255     const _AgreeButton({required this.onTap});
0256
0257     @override
0258     Widget build(BuildContext context) {
0259         return GestureDetector(
0260             onTap: onTap,
0261             child: Container(

```

```

0262         padding:
0263             const EdgeInsets.symmetric(vertical: CyberDimensions.spacingMS),
0264         decoration: BoxDecoration(
0265             gradient: const LinearGradient(
0266                 colors: [CyberColors.neonCyan, CyberColors.neonPurple],
0267                 begin: Alignment.topLeft,
0268                 end: Alignment.bottomRight,
0269             ),
0270             borderRadius: BorderRadius.circular(CyberDimensions.radiusS),
0271         ),
0272         child: Center(
0273             child: Text(
0274                 '同意',
0275                 style: CyberTextStyles.buttonLabel.copyWith(
0276                     color: CyberColors.background,
0277                     fontWeight: FontWeight.w900,
0278                 ),
0279             ),
0280         ),
0281     ),
0282 );
0283 }
0284 }

// ===== 文件结束: lib/features/settings/presentation/widgets/privacy_agreement_modal.dart =====

// ===== 文件: lib/features/settings/providers/settings_provider.dart =====
0001 import 'package:flutter/material.dart';
0002 import 'package:flutter_riverpod/flutter_riverpod.dart';
0004 import '../../../core/database/storage_service.dart';
0005 import '../../../core/utils/cyber_performance_detector.dart';
0006 import '../../../audio/services/ambient_service.dart';
0008 /// 供 Riverpod 使用的全局设置 Provider
0009 final settingsProvider = ChangeNotifierProvider<SettingsProvider>((ref) {
0010     final p = SettingsProvider();
0011     p.loadFromStorage();
0012     return p;
0013 });
0015 /// 全局设置 Provider
0016 /// 完整复刻旧版 main.js 的 `t` 设置对象 + localStorage 持久化逻辑
0017 /// 键名与 JS setting_* 前缀完全对齐
0018 class SettingsProvider with ChangeNotifier {
0019     late bool sound;
0020     late bool storyTts;
0021     late String voice;
0022     late int idleTimeout;
0023     late double ttsRate;
0024     late double ambientVol;
0025     late String ambientStyle;
0027 /// 环境音开关 — 预留字段, 待环境音播放器功能实现后启用
0028 /// 当前无 UI 入口, 也无消费方, 仅持久化存档保持向后兼容

```

```
0029 late bool ambientEnabled;
0031 /// 动画质量设置: 'auto' | 'high' | 'medium' | 'low'
0032 late String animationQualitySetting;
0033 CyberAnimationLevel? _autoDetectedLevel;
0035 /// App 启动时从 StorageService 恢复所有设置
0036 void loadFromStorage() {
0037     sound = StorageService.getSettingSound();
0038     storyTts = StorageService.getSettingStoryTts();
0039     voice = StorageService.getSettingVoice();
0040     // 校验音色有效性, 无效则回退默认 (防止 storage 残留无效音色名)
0041     if (voice != 'zh-CN-XiaoxiaoNeural' &&
0042         voice != 'zh-CN-YunxiNeural' &&
0043         voice != 'zh-CN-YunjianNeural' &&
0044         voice != 'zh-CN-XiaoyiNeural' &&
0045         voice != 'zh-CN-XiaomengNeural') {
0046         voice = 'zh-CN-XiaoxiaoNeural';
0047     }
0048     idleTimeout = StorageService.getSettingIdleTimeout();
0049     ttsRate = StorageService.getSettingTtsRate();
0050     ambientVol = StorageService.getSettingAmbientVol();
0051     ambientEnabled = StorageService.getSettingAmbientEnabled();
0052     ambientStyle = StorageService.getSettingAmbientStyle();
0053     AmbientService.setStyle(ambientStyle);
0054     animationQualitySetting = StorageService.getSettingAnimationQuality();
0055     if (animationQualitySetting == 'auto') {
0056         _autoDetectedLevel = CyberPerformanceDetector.detectLevel();
0057     }
0058     notifyListeners();
0059 }
0061 /// 计算属性: 获取当前界面应当采用的动画等级
0062 CyberAnimationLevel get currentAnimationLevel {
0063     switch (animationQualitySetting) {
0064         case 'high':
0065             return CyberAnimationLevel.high;
0066         case 'medium':
0067             return CyberAnimationLevel.medium;
0068         case 'low':
0069             return CyberAnimationLevel.low;
0070         case 'auto':
0071             return CyberAnimationLevel.low;
0072         default:
0073             return _autoDetectedLevel ?? CyberPerformanceDetector.detectLevel();
0074     }
0075 }
0076 Future<void> setSound(bool v) async {
0077     sound = v;
0078     await StorageService.setSettingSound(v);
0079     notifyListeners();
0080 }
0081 Future<void> setStoryTts(bool v) async {
0082     storyTts = v;
```

```
0086     await StorageService.setSettingStoryTts(v);
0087     notifyListeners();
0088   }
0090   Future<void> setVoice(String v) async {
0091     if (v != 'zh-CN-XiaoxiaoNeural' &&
0092         v != 'zh-CN-YunxiNeural' &&
0093         v != 'zh-CN-YunjianNeural' &&
0094         v != 'zh-CN-XiaoyiNeural' &&
0095         v != 'zh-CN-XiaomengNeural') {
0096       v = 'zh-CN-XiaoxiaoNeural';
0097     }
0098     voice = v;
0099     await StorageService.setSettingVoice(v);
0100     notifyListeners();
0101   }
0103   Future<void> setIdleTimeout(int v) async {
0104     idleTimeout = v;
0105     await StorageService.setSettingIdleTimeout(v);
0106     notifyListeners();
0107   }
0109   Future<void> setTtsRate(double v) async {
0110     ttsRate = v;
0111     await StorageService.setSettingTtsRate(v);
0112     notifyListeners();
0113   }
0115   Future<void> setAmbientVol(double v) async {
0116     ambientVol = v;
0117     await StorageService.setSettingAmbientVol(v);
0118     notifyListeners();
0119   }
0121   Future<void> setAmbientEnabled(bool v) async {
0122     ambientEnabled = v;
0123     await StorageService.setSettingAmbientEnabled(v);
0124     notifyListeners();
0125   }
0127   Future<void> setAmbientStyle(String v) async {
0128     ambientStyle = v;
0129     await StorageService.setSettingAmbientStyle(v);
0130     notifyListeners();
0131   }
0133   Future<void> setAnimationQualitySetting(String v) async {
0134     animationQualitySetting = v;
0135     if (v == 'auto') {
0136       _autoDetectedLevel = CyberPerformanceDetector.detectLevel();
0137     }
0138     await StorageService.setSettingAnimationQuality(v);
0139     notifyListeners();
0140   }
0141 }

// ===== 文件结束: lib/features/settings/providers/settings_provider.dart =====
```

```
// ===== 文件: lib/shared/widgets/cyber_confirm_dialog.dart =====
0001 import 'package:flutter/material.dart';
0002 import 'package:yueyou/core/theme/cyber_colors.dart';
0003 import 'package:yueyou/core/theme/cyber_dimensions.dart';
0004 import 'package:yueyou/core/theme/cyber_text_styles.dart';
0005 import 'package:yueyou/shared/widgets/cyber_modal.dart';
0007 /// 赛博朋克风格确认对话框
0008 /// 用于重置游戏等需要二次确认的操作
0009 /// 基于 showCyberModal 封装, 内部只提供确认内容 (标题 + 消息 + 按钮)
0010 Future<bool> showCyberConfirmDialog({
0011   required BuildContext context,
0012   required String title,
0013   required String message,
0014   String confirmText = '确认',
0015   String cancelText = '取消',
0016 }) async {
0017   final result = await showCyberModal<bool>(<
0018     context: context,
0019     child: Padding(
0020       padding: const EdgeInsets.all(CyberDimensions.spacingL),
0021       child: Column(
0022         mainAxisAlignment: MainAxisAlignment.min,
0023         children: [
0024           Text(
0025             title,
0026             style: CyberTextStyles.dialogTitle.copyWith(
0027               color: CyberColors.whiteHigh,
0028               fontWeight: FontWeight.w900,
0029             ),
0030           ),
0031           const SizedBox(height: CyberDimensions.spacingM),
0032           Flexible(
0033             child: SingleChildScrollView(
0034               child: Text(
0035                 message,
0036                 textAlign: TextAlign.center,
0037                 style: CyberTextStyles.bodySmall.copyWith(
0038                   color: CyberColors.whiteMedium,
0039                   fontSize: 14,
0040                   height: 1.5,
0041                 ),
0042               ),
0043             ),
0044           ),
0045           const SizedBox(height: CyberDimensions.spacingL),
0046           Row(
0047             children: [
0048               Expanded(
0049                 child: _CyberButton(
```

```

0050         text: cancelText,
0051         isPrimary: false,
0052         onTap: () => Navigator.of(context).pop(false),
0053       ),
0054     ),
0055     const SizedBox(width: CyberDimensions.spacingMS),
0056     Expanded(
0057       child: _CyberButton(
0058         text: confirmText,
0059         isPrimary: true,
0060         onTap: () => Navigator.of(context).pop(true),
0061       ),
0062     ),
0063   ],
0064 ),
0065 ],
0066 ),
0067 ),
0068 );
0069 return result ?? false;
0070 }
0071
0072 class _CyberButton extends StatelessWidget {
0073   final String text;
0074   final bool isPrimary;
0075   final VoidCallback onTap;
0076
0077   const _CyberButton({
0078     required this.text,
0079     required this.isPrimary,
0080     required this.onTap,
0081   });
0082
0083   @override
0084   Widget build(BuildContext context) {
0085     return GestureDetector(
0086       onTap: onTap,
0087       child: Container(
0088         padding:
0089           const EdgeInsets.symmetric(vertical: CyberDimensions.spacingMS),
0090         decoration: BoxDecoration(
0091           gradient: isPrimary
0092             ? const LinearGradient(
0093               colors: [CyberColors.hotPink, CyberColors.neonPurple],
0094               begin: Alignment.topLeft,
0095               end: Alignment.bottomRight,
0096             )
0097             : null,
0098           color: isPrimary ? null : CyberColors.whiteSubtle,
0099           borderRadius: BorderRadius.circular(CyberDimensions.radiusS),
0100           border: Border.all(
0101             color:
0102               isPrimary ? CyberColors.transparent : CyberColors.whiteSubtle,

```



```

0103         width: CyberDimensions.borderNormal,
0104     ),
0105 ),
0106 child: Center(
0107     child: Text(
0108         text,
0109         style: CyberTextStyles.buttonLabel.copyWith(
0110             color: CyberColors.whiteHigh,
0111             fontWeight: FontWeight.w900,
0112         ),
0113     ),
0114 ),
0115 ),
0116 );
0117 }
0118 }
// ===== 文件结束: lib/shared/widgets/cyber_confirm_dialog.dart =====

// ===== 文件: lib/shared/widgets/cyber_modal.dart =====
0001 import 'dart:ui';
0002 import 'package:flutter/material.dart';
0003 import 'package:yueyou/core/theme/cyber_colors.dart';
0004 import 'package:yueyou/core/theme/cyber_dimensions.dart';
0005 import 'package:yueyou/core/utils/cyber_performance_detector.dart';
0007 /// 赛博朋克风格全局弹窗封装
0008 /// 替代 Navigator.push 全屏跳转, 点击外部可关闭
0009 Future<T?> showCyberModal<T>({
0010     required BuildContext context,
0011     required Widget child,
0012     bool barrierDismissible = true,
0013 }) {
0014     return showGeneralDialog<T>({
0015         context: context,
0016         barrierDismissible: barrierDismissible,
0017         barrierLabel: MaterialLocalizations.of(context).modalBarrierDismissLabel,
0018         barrierColor: CyberColors.blackOverlay,
0019         transitionDuration: CyberDimensions.animNormal,
0020         pageBuilder: (context, animation, secondaryAnimation) {
0021             return SafeArea(
0022                 child: Center(
0023                     child: Material(
0024                         type: MaterialType.transparency,
0025                         child: ScaleTransition(
0026                             scale: CurvedAnimation(
0027                                 parent: animation,
0028                                 curve: Curves.easeOutCubic,
0029                             ),
0030                         child: FadeTransition(
0031                             opacity: animation,
0032                             child: Container(

```

```

0033     margin: const EdgeInsets.symmetric(
0034       horizontal: CyberDimensions.spacingXL,
0035       vertical: 64,
0036     ),
0037     constraints: BoxConstraints(
0038       maxWidth: 500,
0039       maxHeight: MediaQuery.of(context).size.height * 0.85,
0040     ),
0041     decoration: BoxDecoration(
0042       borderRadius:
0043         BorderRadius.circular(CyberDimensions.radiusL),
0044       border: Border.all(
0045         color: CyberColors.neonCyan.withValues(alpha: 0.4),
0046         width: CyberDimensions.borderThick,
0047       ),
0048       boxShadow: [
0049         BoxShadow(
0050           color: CyberColors.neonCyan.withValues(alpha: 0.25),
0051           blurRadius: 30,
0052           spreadRadius: 2,
0053         ),
0054         const BoxShadow(
0055           color: CyberColors.blackShadow,
0056           blurRadius: 40,
0057           offset: Offset(0, 15),
0058         ),
0059       ],
0060     ),
0061     child: ClipRRect(
0062       borderRadius: BorderRadius.circular(
0063         CyberDimensions.radiusL - CyberDimensions.borderThick,
0064       ),
0065       child: CyberPerformanceDetector.detectLevel() ==
0066         CyberAnimationLevel.low
0067         ? Container(
0068           color:
0069             CyberColors.glassDark.withValues(alpha: 0.98),
0070           child: child, // 直接渲染内容
0071         )
0072         : BackdropFilter(
0073           filter: ImageFilter.blur(
0074             sigmaX: CyberDimensions.blurStrong,
0075             sigmaY: CyberDimensions.blurStrong,
0076           ),
0077           child: Container(
0078             color: CyberColors.glassDark,
0079             child: child, // 移除了冗余的内部 ClipRRect, 统一在外层精准截断
0080           ),
0081         ),
0082     ),

```

```
0083         ),
0084     ),
0085     ),
0086     ),
0087     ),
0088     );
0089   },
0090 );
0091 }

// ===== 文件结束: lib/shared/widgets/cyber_modal.dart =====

// ===== 文件: lib/shared/widgets/cyber_toast.dart =====
0001 import 'dart:async';
0002 import 'package:flutter/material.dart';
0003 import 'dart:ui';
0004 import '../core/theme/cyber_colors.dart';
0005 import '../core/theme/cyber_dimensions.dart';
0006 import '../core/theme/cyber_text_styles.dart';
0007 import '../core/utils/cyber_performance_detector.dart';
0008 import '../features/game_2048/presentation/widgets/board_mascot.dart';
0009 import '../main.dart';
0010
0011 enum ToastType {
0012   info,
0013   error,
0014   success,
0015 }
0016
0017 class CyberToast {
0018   static OverlayEntry? _currentEntry;
0019   static Timer? _currentTimer;
0020
0021   static void show(
0022     String message, {
0023     ToastType type = ToastType.info,
0024     Duration duration = CyberDimensions.toastDuration,
0025   }) {
0026     // 移除之前的 Toast
0027     _removeCurrentEntry();
0028     // 创建新的 OverlayEntry
0029     final overlay = globalNavigatorKey.currentState?.overlay;
0030     if (overlay == null) return;
0031     final entry = OverlayEntry(
0032       builder: (context) => _CyberToastWidget(
0033         message: message,
0034         type: type,
0035         onRemove: () => _currentEntry = null,
0036       ),
0037     );
0038     overlay.insert(entry);
0039     _currentEntry = entry;
0040     overlay.insert(entry);
0041     // 设置定时器, 自动移除
0042     _currentTimer?.cancel();
0043     _currentTimer = Timer(duration, () {
0044       _currentEntry?.remove();
0045     });
0046   }
0047 }
```

```

0045     _currentTimer = Timer(duration, () {
0046         _removeCurrentEntry();
0047     });
0048 }
0050 static void _removeCurrentEntry() {
0051     if (_currentEntry != null) {
0052         _currentEntry?.remove();
0053         _currentEntry = null;
0054     }
0055     _currentTimer?.cancel();
0056     _currentTimer = null;
0057 }
0058 }
0060 class _CyberToastWidget extends StatefulWidget {
0061     final String message;
0062     final ToastType type;
0063     final VoidCallback onRemove;
0065     const _CyberToastWidget({
0066         required this.message,
0067         required this.type,
0068         required this.onRemove,
0069     });
0071     @override
0072     __CyberToastWidgetState createState() => __CyberToastWidgetState();
0073 }
0075 class __CyberToastWidgetState extends State<_CyberToastWidget>
0076     with SingleTickerProviderStateMixin {
0077     late AnimationController _controller;
0078     late Animation<Offset> _slideAnimation;
0079     late Animation<double> _fadeAnimation;
0081     @override
0082     void initState() {
0083         super.initState();
0085         _controller = AnimationController(
0086             duration: CyberDimensions.animNormal,
0087             vsync: this,
0088         );
0090         _slideAnimation = Tween<Offset>()
0091             .begin(const Offset(0, -1),
0092             end: Offset.zero,
0093         ).animate(
0094             CurvedAnimation(
0095                 parent: _controller,
0096                 curve: Curves.easeOut,
0097             ),
0098         );
0100         _fadeAnimation = Tween<double>()
0101             .begin(0.0,
0102             end: 1.0,
0103         ).animate(

```

```

0104     CurvedAnimation(
0105         parent: _controller,
0106         curve: Curves.easeOut,
0107     ),
0108 );
0110 _controller.forward();
0111 }
0113 @override
0114 void dispose() {
0115     _controller.dispose();
0116     super.dispose();
0117 }
0119 Color _getBorderColor() => switch (widget.type) {
0120     ToastType.error => CyberColors.neonPink,
0121     ToastType.success => CyberColors.neonGreen,
0122     ToastType.info => CyberColors.neonCyan
0123 };
0125 @override
0126 Widget build(BuildContext context) {
0127     return Positioned(
0128         top: MediaQuery.of(context).padding.top + CyberDimensions.spacingM,
0129         left: 0,
0130         right: 0,
0131         child: FadeTransition(
0132             opacity: _fadeAnimation,
0133             child: SlideTransition(
0134                 position: _slideAnimation,
0135                 child: Align(
0136                     alignment: Alignment.topCenter,
0137                     child: IntrinsicWidth(
0138                         child: Container(
0139                             constraints: BoxConstraints(
0140                                 maxWidth: MediaQuery.of(context).size.width * 0.85,
0141                             ),
0142                             margin: const EdgeInsets.symmetric(
0143                                 horizontal: CyberDimensions.spacingML,
0144                             ),
0145                             decoration: BoxDecoration(
0146                                 borderRadius: BorderRadius.circular(CyberDimensions.radiusL),
0147                                 border: Border.all(
0148                                     color: _getBorderColor(),
0149                                     width: CyberDimensions.borderThick,
0150                                 ),
0151                                 boxShadow: [
0152                                     BoxShadow(
0153                                         color: _getBorderColor().withValues(alpha: 0.3),
0154                                         blurRadius: 16,
0155                                         offset: const Offset(0, 4),
0156                                     ),
0157                                 ],

```

```

0158         ),
0159         child: ClipRRect(
0160           borderRadius: BorderRadius.circular(
0161             CyberDimensions.radiusL - CyberDimensions.borderThick,
0162           ),
0163           child: CyberPerformanceDetector.detectLevel() ==
0164             CyberAnimationLevel.low
0165             ? Container(
0166               color: CyberColors.glassDark.withValues(alpha: 0.98),
0167               child: _buildContent(),
0168             )
0169             : BackdropFilter(
0170               filter: ImageFilter.blur(sigmaX: 10, sigmaY: 10),
0171               child: Container(
0172                 color: CyberColors.glassDark,
0173                 child: _buildContent(),
0174               ),
0175             ),
0176       ),
0177     ),
0178   ),
0179 ),
0180 ),
0181 ),
0182 );
0183 }
0184
0185 Widget _buildContent() {
0186   return Padding(
0187     padding: const EdgeInsets.symmetric(
0188       horizontal: 16,
0189       vertical: 12,
0190     ),
0191     child: Row(
0192       mainAxisAlignment: MainAxisAlignment.min,
0193       crossAxisAlignment: CrossAxisAlignment.center,
0194       children: [
0195         // XIAOYO Avatar PFP
0196         Container(
0197           width: 44,
0198           height: 44,
0199           decoration: BoxDecoration(
0200             shape: BoxShape.circle,
0201             color: CyberColors.background.withValues(alpha: 0.8),
0202             border: Border.all(
0203               color: _getBorderColor().withValues(alpha: 0.7),
0204               width: 1,
0205             ),
0206             boxShadow: [
0207               BoxShadow(
0208                 color: _getBorderColor().withValues(alpha: 0.5),

```

```

0209         blurRadius: 8,
0210     ),
0211 ],
0212 ),
0213 child: ClipOval(
0214     child: Transform.scale(
0215         scale: 0.65,
0216         // Transform.translate 去微调位置以确保脸部在圆圈正中心
0217         child: Transform.translate(
0218             offset: const Offset(0, -5),
0219             child: const IgnorePointer(
0220                 child: BoardMascot(),
0221             ),
0222         ),
0223     ),
0224 ),
0225 ),
0226 const SizedBox(width: 14),
0227 // Text Section
0228 Flexible(
0229     child: Column(
0230         mainAxisAlignment: MainAxisAlignment.min,
0231         crossAxisAlignment: CrossAxisAlignment.start,
0232         children: [
0233             Text(
0234                 'XIAOYU',
0235                 style: CyberTextStyles.captionBold.copyWith(
0236                     color: _getBorderColor(),
0237                     fontSize: 10,
0238                     letterSpacing: 1.2,
0239                 ),
0240             ),
0241             const SizedBox(height: 4),
0242             Text(
0243                 widget.message,
0244                 style: CyberTextStyles.bodySmallBold.copyWith(
0245                     color: CyberColors.whiteHigh,
0246                     height: 1.3,
0247                 ),
0248                 softWrap: true,
0249             ),
0250         ],
0251     ),
0252 ),
0253 ],
0254 ),
0255 );
0256 }
0257 }

```

// ===== 文件结束: lib/shared/widgets/cyber_toast.dart =====

```
// ===== 文件: lib/shared/widgets/safe_padding_wrap.dart =====
0001 import 'package:flutter/material.dart';
0002 import 'package:yueyou/core/theme/cyber_colors.dart';
0003 import 'package:yueyou/core/utils/safe_area_utils.dart';
0005 /// 顶部安全区包裹组件
0006 /// 严格执行“呼吸感”准则: SafeArea + 额外 24.0px 的 Padding
0007 class SafePaddingWrap extends StatelessWidget {
0008   final Widget child;
0010   const SafePaddingWrap({
0011     super.key,
0012     required this.child,
0013   });
0015   @override
0016   Widget build(BuildContext context) {
0017     return Container(
0018       color: CyberColors.background,
0019       child: SafeArea(
0020         bottom: false, // 顶部组件不强制底部安全区
0021         child: Padding(
0022           padding: const EdgeInsets.only(
0023             top: SafeAreaUtils.topBreathPadding,
0024             left: 16.0,
0025             right: 16.0,
0026           ),
0027           child: child,
0028         ),
0029       ),
0030     );
0031   }
0032 }

// ===== 文件结束: lib/shared/widgets/safe_padding_wrap.dart =====

// ===== 文件: lib/shared/widgets/tts_error_listener.dart =====
0001 import 'package:flutter/material.dart';
0002 import 'package:yueyou/core/utils/cyber_logger.dart';
0003 import 'package:flutter_riverpod/flutter_riverpod.dart';
0004 import '../features/audio/domain/tts_audio_state.dart';
0005 import '../features/audio/providers/tts_audio_notifier.dart';
0006 import 'cyber_toast.dart';
0008 /// TTS 错误全局监听器组件
0009 ///
0010 /// 将任意页面或整个 App 根节点包裹在此组件下,
0011 /// 每当 [TtsAudioError] 产生新时间戳时,
0012 /// 自动弹出 [CyberToast] 错误提示, 无需各页面手动监听。
0013 class TtsErrorListener extends ConsumerStatefulWidget {
0014   final Widget child;
0015   const TtsErrorListener({super.key, required this.child});
0017   @override
0018   ConsumerState<TtsErrorListener> createState() => _TtsErrorListenerState();
```



```

0019 }
0021 class _TtsErrorListenerState extends ConsumerState<TtsErrorListener> {
0022     int _previousErrorTime = 0;
0023     String? _previousFallback;
0024     int _lastFallbackTimestamp = 0;
0026     @override
0027     void didChangeDependencies() {
0028         super.didChangeDependencies();
0029         // 依赖变化时刷新快照，避免初始化重复触发旧错误
0030         final audioState = ref.read(ttsAudioProvider);
0031         _previousErrorTime = switch (audioState) {
0032             TtsAudioError(:final timestamp) => timestamp,
0033             TtsAudioIdle() ||
0034             TtsAudioBuffering() ||
0035             TtsAudioPlaying() ||
0036             TtsAudioPaused() =>
0037                 0,
0038         };
0039         _previousFallback = audioState.fallbackMessage;
0040     }
0042     @override
0043     Widget build(BuildContext context) {
0044         final audioState = ref.watch(ttsAudioProvider);
0045         final (:err, :timestamp) = switch (audioState) {
0046             TtsAudioError(:final message, :final timestamp) => (
0047                 err: message,
0048                 timestamp: timestamp,
0049             ),
0050             TtsAudioIdle() ||
0051             TtsAudioBuffering() ||
0052             TtsAudioPlaying() ||
0053             TtsAudioPaused() =>
0054                 (err: null, timestamp: 0),
0055         };
0056         final currentTimestamp = DateTime.now().millisecondsSinceEpoch;
0057         if (err != null && timestamp != _previousErrorTime) {
0058             _previousErrorTime = timestamp;
0059             CyberLogger.captureMessage('[TtsErrorListener] 检测到新错误，将展示 CyberToast');
0060             WidgetsBinding.instance.addPostFrameCallback(() {
0061                 if (!mounted) return;
0062                 try {
0063                     CyberToast.show(err, type: ToastType.error);
0064                 } catch (e, stack) {
0065                     CyberLogger.captureWarning(
0066                         e,
0067                         stack: stack,
0068                         tag: 'tts',
0069                         extra: {'context': 'TtsErrorListener 展示错误 Toast 失败'},
0070                     );
0071                 }
0072             });
0073         }
0074     }

```

```

0073     });
0074   }
0075   final fallback = audioState.fallbackMessage;
0076   if (fallback == null) {
0077     _previousFallback = null;
0078   } else {
0079     final isSameFallback = fallback == _previousFallback;
0080     final isThrottled =
0081       isSameFallback && (currentTimestamp - _lastFallbackTimestamp < 1000);
0082     if (!isThrottled) {
0083       _previousFallback = fallback;
0084       _lastFallbackTimestamp = currentTimestamp;
0085       WidgetsBinding.instance.addPostFrameCallback((_) {
0086         if (!mounted) return;
0087         try {
0088           CyberToast.show(fallback, type: ToastType.info);
0089         } catch (e, stack) {
0090           CyberLogger.captureWarning(
0091             e,
0092             stack: stack,
0093             tag: 'tts',
0094             extra: {'context': 'TtsErrorListener 展示降级通知 Toast 失败'},
0095           );
0096         }
0097       });
0098     }
0099   });
0100 }
0101 }
0102
0103 return widget.child;
0104 }
0105 }

```

// ===== 文件结束: lib/shared/widgets/tts_error_listener.dart =====

// ===== 文件: server/main.go =====

```

0001 package main
0002 import (
0003     "log"
0004     "github.com/gin-gonic/gin"
0005 )
0006 func main() {
0007     gin.SetMode(gin.ReleaseMode)
0008     r := gin.Default()
0009     if err := r.SetTrustedProxies(nil); err != nil {
0010         log.Fatal(err)
0011     }
0012     // —— TTS 语音合成 ——
0013     r.POST("/api/v1/tts", ttsHandler)
0014     // —— 书籍服务 ——
0015     r.GET("/api/v1/book/catalog", bookCatalogHandler)
0016     r.POST("/api/v1/book/chapter", bookChapterHandler)
0017     // —— 合规页面 ——

```

```
0024     r.GET("/privacy", privacyHandler)
0026     log.Println("yueyou-server on :8081")
0027     r.Run(":8081")
0028 }
// ===== 文件结束: server/main.go =====

// ===== 文件: server/config.go =====
0001 package main
0003 import "os"
0005 // Config 持有所有运行时配置, 优先从环境变量读取, 回退到内置默认值。
0006 // 生产部署: 通过 systemd EnvironmentFile 或 Docker --env-file 注入。
0007 type Config struct {
0008     AKID      string // 阿里云 AccessKey ID
0009     AKSecret   string // 阿里云 AccessKey Secret
0010     OSSEndpoint string // OSS 内网端点 (服务器与 OSS 同 region 时用内网省流量)
0011     OSSBucket  string // OSS Bucket 名称
0012     OSSPub     string // OSS / CDN 公网访问前缀 (无尾斜线)
0013 }
0015 // cfg 为全局单例配置, 在 main() 启动前由 init() 初始化。
0016 var cfg Config
0018 func init() {
0019     cfg = Config{
0020         AKID:      getenv("YUEYOU_AKID", ""),
0021         AKSecret:   getenv("YUEYOU_AKSEC", ""),
0022         OSSEndpoint: getenv("YUEYOU_OSS_EP", "oss-cn-beijing-internal.aliyuncs.com"),
0023         OSSBucket:  getenv("YUEYOU_OSS_BKT", "general-storage"),
0024         OSSPub:     getenv("YUEYOU_OSS_PUB", "https://general-storage.oss-cn-beijing.aliyuncs.com")
0025     }
0026 }
0028 func getenv(key, fallback string) string {
0029     if v := os.Getenv(key); v != "" {
0030         return v
0031     }
0032     return fallback
0033 }
// ===== 文件结束: server/config.go =====

// ===== 文件: server/handler_tts.go =====
0001 package main
0003 import (
0004     "bytes"
0005     "crypto/md5"
0006     "fmt"
0007     "log"
0008     "os"
0009     "os/exec"
0010     "sync"
0012     "github.com/aliyun/aliyun-oss-go-sdk/oss"
0013     "github.com/gin-gonic/gin"
0014 )
```

```

0016 // edgeTtsSem 严格串行化 edge-tts 进程，避免触发微软服务端并发任务限制。
0017 var edgeTtsSem = make(chan struct{}, 1)
0019 // inflightMu 保护 inflightKeys，用于对同一文本的并发请求去重。
0020 var (
0021     inflightMu sync.Mutex
0022     inflightKeys = map[string]chan struct{} {}
0023 )
0025 // ttsHandler 接受文本和音色，生成 MP3 并上传 OSS，返回 CDN 播放地址。
0026 // 遵循分离下载原则：客户端 POST 获取 URL，再自行 GET 下载音频。
0027 func ttsHandler(c *gin.Context) {
0028     var req struct {
0029         Text string `json:"text" binding:"required"`
0030         Voice string `json:"voice"`
0031     }
0032     if err := c.ShouldBindJSON(&req); err != nil {
0033         c.JSON(400, gin.H{"status": "error", "message": "JSON error"})
0034         return
0035     }
0036     if req.Voice == "" {
0037         req.Voice = "zh-CN-XiaoxiaoNeural"
0038     }
0040     hash := md5.Sum([]byte(req.Text + req.Voice))
0041     fn := fmt.Sprintf("cache/%x.mp3", hash)
0042     fu := fmt.Sprintf("%s/%s", cfg.OSSPub, fn)
0044     bk, err := ossBucket()
0045     if err != nil {
0046         c.JSON(500, gin.H{"status": "error", "message": "OSS 初始化失败"})
0047         return
0048     }
0050     // 缓存命中直接返回
0051     if ok, _ := bk.IsObjectExist(fn); ok {
0052         c.JSON(200, gin.H{"status": "success", "url": fu})
0053         return
0054     }
0056     // 同一文本正在合成时，等待已有任务完成后复用 OSS 缓存，避免重复调用 edge-tts
0057     inflightMu.Lock()
0058     if ch, ok := inflightKeys[fn]; ok {
0059         inflightMu.Unlock()
0060         <-ch
0061         if ok2, _ := bk.IsObjectExist(fn); ok2 {
0062             c.JSON(200, gin.H{"status": "success", "url": fu})
0063         } else {
0064             c.JSON(500, gin.H{"status": "error", "message": "语音合成失败"})
0065         }
0066         return
0067     }
0068     doneCh := make(chan struct{})
0069     inflightKeys[fn] = doneCh
0070     inflightMu.Unlock()
0071     defer func() {

```

```

0072         close(doneCh)
0073         inflightMu.Lock()
0074         delete(inflightKeys, fn)
0075         inflightMu.Unlock()
0076     }()
0077     log.Printf("[TTS] 合成: %s", req.Text)
0078     // 获取信号量, 严格串行化 edge-tts 进程
0079     edgeTtsSem <- struct{}{}
0080     defer func() { <-edgeTtsSem }()
0081     tmpFile := fmt.Sprintf("/tmp/tts_%x.mp3",
0082         md5.Sum([]byte(req.Text+req.Voice+fmt.Sprintf("%d", os.Getpid()))))
0083     cmd := exec.Command("edge-tts",
0084         "--text", req.Text,
0085         "--voice", req.Voice,
0086         "--write-media", tmpFile,
0087     )
0088     if out, err := cmd.CombinedOutput(); err != nil {
0089         log.Printf("[TTS] edge-tts 错误: %v, output: %s", err, string(out))
0090         c.JSON(500, gin.H{"status": "error", "message": "语音合成失败"})
0091         return
0092     }
0093     audioData, err := os.ReadFile(tmpFile)
0094     os.Remove(tmpFile)
0095     if err != nil {
0096         c.JSON(500, gin.H{"status": "error", "message": "读取音频失败"})
0097         return
0098     }
0099     if len(audioData) < 100 {
0100         c.JSON(500, gin.H{"status": "error", "message": "生成音频为空"})
0101         return
0102     }
0103     if err := bk.PutObject(fn, bytes.NewReader(audioData)); err != nil {
0104         c.JSON(500, gin.H{"status": "error", "message": "OSS 上传失败"})
0105         return
0106     }
0107     c.JSON(200, gin.H{"status": "success", "url": fu})
0108 }
0109 // ossBucket 返回配置好的 OSS bucket 客户端。
0110 func ossBucket() (*oss.Bucket, error) {
0111     cli, err := oss.New(cfg.OSSEndpoint, cfg.AKID, cfg.AKSecret)
0112     if err != nil {
0113         return nil, err
0114     }
0115     return cli.Bucket(cfg.OSSBucket)
0116 }
0117 // ===== 文件结束: server/handler_tts.go =====
0118
0119 // ===== 文件: server/handler_book.go =====
0120 package main
0121 import (

```

```

0004     _ "embed"
0005     "fmt"
0006     "log"
0008     "github.com/gin-gonic/gin"
0009 )
0011 // xiyoujiCatalogJSON 在编译期嵌入目录文件, 换版本时只需替换 data/xiyouji_catalog.json 并重新编译。
0012 //
0013 //go:embed data/xiyouji_catalog.json
0014 var xiyoujiCatalogJSON []byte
0016 // xiyoujiTotalChapters 总章节数, 与客户端 BookConstants.defaultTotalChapters 严格对齐。
0017 const xiyoujiTotalChapters = 100
0019 // bookCatalogHandler 返回西游记目录(章节标题列表)。
0020 //
0021 // GET /api/v1/book/catalog?bookId=xiyouji
0022 //
0023 // 响应格式: {"status": "success", "chapters": [{"title": "...", "lineIndex": 0} ...]}
0024 func bookCatalogHandler(c *gin.Context) {
0025     bookID := c.Query("bookId")
0026     if bookID != "xiyouji" {
0027         c.JSON(404, gin.H{"status": "error", "message": "书籍不存在"})
0028         return
0029     }
0030     // 直接返回嵌入的 JSON, 客户端按约定解析
0031     c.Data(200, "application/json; charset=utf-8", xiyoujiCatalogJSON)
0032 }
0034 // bookChapterHandler 派发章节 CDN 下载地址, 遵循分离下载原则。
0035 //
0036 // POST /api/v1/book/chapter
0037 // 请求体: {"bookId": "xiyouji", "chapterIndex": 0}
0038 // 响应体: {"status": "success", "url": "https://cdn.../books/chapters/xiyouji/001.txt"}
0039 //
0040 // 客户端收到 URL 后自行 GET 下载章节正文, 与 TTS 分离下载完全对齐。
0041 func bookChapterHandler(c *gin.Context) {
0042     var req struct {
0043         BookID      string `json:"bookId"      binding:"required"`
0044         ChapterIndex int    `json:"chapterIndex"`
0045     }
0046     if err := c.ShouldBindJSON(&req); err != nil {
0047         c.JSON(400, gin.H{"status": "error", "message": "参数错误"})
0048         return
0049     }
0050     if req.BookID != "xiyouji" {
0051         c.JSON(404, gin.H{"status": "error", "message": "书籍不存在"})
0052         return
0053     }
0054     if req.ChapterIndex < 0 || req.ChapterIndex >= xiyoujiTotalChapters {
0055         c.JSON(400, gin.H{"status": "error", "message": "章节索引越界"})
0056         return
0057     }
0059     // chapterIndex 0-based, OSS 文件名 1-based 三位补零

```

```
0060     filename := fmt.Sprintf("%03d.txt", req.ChapterIndex+1)
0061     url := fmt.Sprintf("%s/books/xiyouji/%s", cfg.OSSPub, filename)
0063     log.Printf("[Book] 派发章节: index=%d -> %s", req.ChapterIndex, url)
0065     c.JSON(200, gin.H{
0066         "status": "success",
0067         "url":     url,
0068     })
0069 }
```

// ===== 文件结束: server/handler_book.go =====

// ===== 文件: server/handler_privacy.go =====

```
0001 package main
0002 import (
0003     "net/http"
0004     "github.com/gin-gonic/gin"
0005 )
0006 const privacyPolicyHTML = `<!doctype html>
0007 <html lang="zh-CN">
0008 <head>
0009     <meta charset="utf-8">
0010     <meta name="viewport" content="width=device-width, initial-scale=1">
0011     <title>阅游隐私政策</title>
0012     <style>
0013         body { margin: 0; font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
0014             main { max-width: 880px; margin: 0 auto; padding: 32px 18px 64px; }
0015             h1, h2 { color: #22d3ee; }
0016             h1 { border-bottom: 1px solid rgba(34, 211, 238, .35); padding-bottom: 12px; }
0017             section { margin-top: 28px; padding: 18px; border: 1px solid rgba(34, 211, 238, .18); border-
0018                 a { color: #67e8f9; }
0019                 .muted { color: rgba(229, 250, 255, .72); }
0020     </style>
0021 </head>
0022 <body>
0023 <main>
0024     <h1>阅游隐私政策</h1>
0025     <p class="muted">更新日期: 2026年5月4日 | 生效日期: 2026年5月4日</p>
0026     <section>
0027         <h2>一、开发者信息</h2>
0028         <p>开发者/运营主体: 胡传龙</p>
0029         <p>联系邮箱: <a href="mailto:hucloong@163.com">hucloong@163.com</a></p>
0030     </section>
0031     <section>
0032         <h2>二、我们如何收集和使用信息</h2>
0033         <p>阅游是一款本地小说阅读、听读与 2048 益智游戏应用。我们不会收集通讯录、短信、通话记录、定位、相机、麦克风等敏感个人信息。</p>
0034         <p>当您使用云端 TTS 朗读功能时, 应用会将当前需要朗读的文本片段发送至 TTS 服务, 用于生成音频。除当前朗读文本外, 我们不会上传您的书
0035     </section>
0036     <section>
0037         <h2>三、本地数据存储</h2>
0038         <p>阅读进度、书架、设置、游戏存档和 TTS 缓存文件仅保存在您的设备本地。您可以通过卸载应用或清除应用数据删除这些本地数据。</p>
0039     </section>
0040 </body>
0041 </html>`
```

```

0047 <section>
0048     <h2>四、文件访问说明</h2>
0049     <p>导入 TXT 小说时，阅游通过 Android 系统文件选择器访问您主动选择的文件。应用不会扫描您的其他目录，也不会未经您选择的情况下读取其
0050 </section>
0052 <section>
0053     <h2>五、第三方服务</h2>
0054     <p>阅游可能使用云端 TTS 服务、对象存储/CDN 服务提供音频合成、默认书籍目录与章节下载能力。这些服务仅用于实现应用功能，不用于个人画像
0055 </section>
0057 <section>
0058     <h2>六、权限使用说明</h2>
0059     <p>网络权限用于访问 TTS 与书籍服务。文件导入通过系统文件选择器完成授权，不申请读取全部存储空间权限。</p>
0060 </section>
0062 <section>
0063     <h2>七、未成年人保护</h2>
0064     <p>阅游不面向低龄儿童提供定向服务。如未成年人使用本应用，应在监护人指导下使用。</p>
0065 </section>
0067 <section>
0068     <h2>八、您的权利</h2>
0069     <p>如您需要咨询隐私问题、反馈问题或请求删除相关数据，可通过邮箱 <a href="mailto:hucloong@163.com">hucloong@163.com</a> 与
0070 </section>
0072 <section>
0073     <h2>九、政策更新</h2>
0074     <p>我们可能根据功能变化更新本隐私政策。更新后将通过页面日期或应用内提示展示。</p>
0075 </section>
0076 </main>
0077 </body>
0078 </html>`
0080 // privacyHandler 返回阅游隐私政策静态页面。
0081 func privacyHandler(c *gin.Context) {
0082     c.Data(http.StatusOK, "text/html; charset=utf-8", []byte(privacyPolicyHTML))
0083 }
// ===== 文件结束: server/handler_privacy.go =====

//
//
//
//
//
//
//
//
//
//
//
//
//
//
//
//
//
//
//
//
//

```


